

Prototipo funcional de un sistema inteligente de seguimiento y alerta para activos financieros

Ing. María Fabiana Cid

Carrera de Especialización en Inteligencia Artificial

Director: Dr. Camilo Argoty Pulido (FIUBA)

Co-Director: Ph.D. Luciano Machain (UNR)

Jurados:

Esp. Ing. Federico Arias Suarez (FIUBA)

Esp. Ing. Juan Pablo Alianak (FIUBA)

Ing. Christopher Charaf (URBC)

Ciudad de Rosario, abril de 2026

Resumen

En la presente memoria se realiza una descripción del prototipo funcional de un sistema inteligente de seguimiento y alerta de activos financieros. El objetivo es asistir a inversores minoristas no especializados en finanzas en la toma de decisiones respecto a sus activos financieros. El sistema, basado en multiagentes, logra ofrecer predicciones explicables, recomendaciones en lenguaje natural y alertas configurables según el perfil de riesgo de cada usuario. De este modo, el trabajo integra conocimientos de aprendizaje automático, procesamiento natural del lenguaje, ingeniería de software, visualización de datos y seguridad. Si bien es un emprendimiento personal, podría aplicarse en el futuro en el sector financiero.

Agradecimientos

Quiero agradecer a mi esposo Eduardo y a mis cuatro hijos Francisco, Manuel, Inés y Victoria por su paciencia y por su aliento. Agradezco también muy especialmente a mi amiga Belén por haberme inspirado para realizar este posgrado. Finalmente, muchas gracias a mis correctores, a mi director, mi codirector y mis jurados por el tiempo y la enseñanza brindada.

Índice general

Resumen	I
1. Introducción general	1
1.1. Contexto financiero y tecnológico	1
1.2. Estado del arte	2
1.3. Motivación	3
1.4. Objetivos y alcance	4
1.4.1. Objetivo general	4
1.4.2. Objetivos específicos	4
1.4.3. Alcance	5
1.5. Impacto social y económico	5
2. Introducción específica	7
2.1. Sistema multiagente	7
2.2. Machine learning aplicado a finanzas	8
2.3. Arquitectura general	9
2.3.1. Diagrama en bloques del sistema	9
2.3.2. Pipeline secuencial y modularidad	10
2.4. Evaluación de riesgos y explicabilidad	10
2.4.1. Riesgos en predicciones financieras	10
2.4.2. Explicabilidad	11
2.4.3. Riesgo operacional y robustez del modelo	11
2.4.4. Cumplimiento regulatorio: Basilea III y BCRA comunicación A 7724	12
3. Diseño e implementación	13
3.1. Arquitectura multiagente	13
3.2. Backend basado en FastAPI	16
3.3. Interfaz en Streamlit	18
3.4. Base de datos	20
3.5. Sistema de alertas	22
3.6. Seguridad	24
3.7. Pipeline de procesamiento	27
3.8. Despliegue y entorno	29
3.9. Monitoreo y logging	31
4. Ensayos y resultados	33
4.1. Validación integral del sistema propuesto	33
4.2. Evaluación de modelos ML	34
4.2.1. Metodología de evaluación	34
4.2.2. Configuración de modelos	34
4.2.3. Resultados de evaluación individual	35
4.2.4. Análisis por ticker	35

4.2.5.	Validación cruzada	36
4.3.	Evaluación NLP	37
4.3.1.	Arquitectura del SentimentAgent	37
4.3.2.	Resultados del SentimentAgent	37
4.3.3.	Análisis de componentes	38
4.3.4.	Correlación entre sentimiento y movimiento de precio	38
4.3.5.	Volumen de noticias procesadas	39
4.3.6.	Rango de scores de sentimiento	40
4.4.	Pruebas end-to-end	40
4.4.1.	Configuración de pruebas	40
4.4.2.	Resultados de pruebas funcionales	40
4.4.3.	Desglose de latencia por ticker	41
4.4.4.	Pruebas de rendimiento bajo carga	42
4.4.5.	Validación de requisitos no funcionales	42
4.4.6.	Evidencia visual del flujo end-to-end	43
4.5.	Casos de uso	44
4.5.1.	Caso de uso 1: inversor principiante.	44
4.5.2.	Caso de uso 2: trader experimentado.	45
4.5.3.	Caso de uso 3: gestor de portafolio.	45
4.5.4.	Síntesis de casos de uso	46
5.	Conclusiones	47
5.1.	Conclusiones generales	47
5.2.	Trabajo futuro	47
A.	Comparación de objetivos planificados vs. realizados	49
	Bibliografía	51

Índice de figuras

1.1. Flujo de información financiera.	2
1.2. Brecha informativa entre grandes inversores y pequeños inversionistas.	5
2.1. Interacción entre agentes en el sistema multiagente.	8
2.2. Arquitectura general del sistema multiagente.	10
3.1. Diagrama vertical de interconexión entre agentes del sistema multiagente.	16
3.2. Diagrama vertical del flujo de peticiones API.	17
3.3. Pantalla principal del dashboard Streamlit: formulario de autenticación e inicio de sesión.	19
3.4. Flujo de comunicación entre el dashboard Streamlit y los tres endpoints principales del backend.	19
3.5. Diagrama entidad-relación de la base de datos.	22
3.6. Diagrama vertical del flujo de generación y consulta de alertas.	23
3.7. Diagrama del flujo de seguridad implementado en el sistema.	26
3.8. Diagrama vertical del pipeline completo de procesamiento, desde la ingesta hasta la visualización.	28
4.1. Cumplimiento de requisitos no funcionales del sistema.	42
4.2. Dashboard Streamlit tras autenticación exitosa: panel de control lateral y área principal en espera de análisis.	43
4.3. Resultado del análisis de AAPL: gráfico de evolución de precio, proyección del modelo y alerta generada por el sistema.	44

Índice de tablas

1.1. Plataformas existentes vs. el prototipo propuesto.	3
3.1. Funciones y salidas de los agentes del sistema.	15
3.2. Estructura de la tabla <code>usuarios</code>	20
3.3. Estructura de la tabla <code>alertas</code>	20
3.4. Estructura de la tabla <code>metricas_modelo</code>	21
3.5. Ejemplo de alertas generadas por movimientos fuera del rango esperado	24
3.6. Componentes de seguridad implementados.	26
3.7. Bibliotecas principales.	29
4.1. Métricas de clasificación del ensemble de modelos.	35
4.2. Rendimiento del ensemble por ticker.	36
4.3. Resultados de validación cruzada temporal del ensemble.	36
4.4. Scores de sentimiento por ticker (13 de febrero de 2026).	38
4.5. Modelos NLP del ensemble <code>Sentimentagent</code>	38
4.6. Sentimiento y variación de precio por ticker (13 de febrero de 2026).	39
4.7. Características del procesamiento de noticias (13 de febrero de 2026).	39
4.8. Resultados de pruebas funcionales (21 de marzo de 2026).	40
4.9. Análisis de latencia por iteración (21 de marzo de 2026).	41
4.10. Latencia total por ticker (21 de marzo de 2026, promedio de 3 iteraciones).	41
4.11. Pruebas de carga concurrente (21 de marzo de 2026).	42
4.12. Señales del sistema por ticker (13 de febrero de 2026).	46
4.13. Resumen de escenarios de uso ilustrativos.	46
A.1. Comparación.	49

***Dedicado a mi esposo Eduardo y a mis hijos Francisco,
Manuel, Inés y Victoria.***

Capítulo 1

Introducción general

El presente capítulo introduce el desarrollo de un sistema multiagente inteligente orientado al seguimiento y alerta de activos financieros. Se destaca la transformación del sector financiero impulsada por la digitalización y la inteligencia artificial (IA), en el que el acceso rápido a la información es clave para la toma de decisiones. Las soluciones actuales, como Bloomberg [1, 2] o FinBERT [3, 4, 5], resultan costosas y poco accesibles para inversionistas minoristas.

El trabajo realizado busca reducir la brecha tecnológica y ofrecer una herramienta autónoma, modular y de bajo costo que integre análisis predictivo, procesamiento de noticias y generación de alertas. Su arquitectura cooperativa de agentes permite escalabilidad y personalización del sistema. El impacto esperado es la democratización del acceso a la información financiera y la promoción de decisiones más informadas y equitativas.

1.1. Contexto financiero y tecnológico

El ecosistema financiero actual atraviesa un proceso de transformación acelerado impulsado por la digitalización, la disponibilidad masiva de datos y los avances en IA. En este entorno, el acceso oportuno a la información y la capacidad de procesarla en tiempo real se han convertido en factores críticos para la toma de decisiones de inversión.

Al mismo tiempo, los mercados financieros presentan alta volatilidad, asimetrías informativas y reacciones tardías ante eventos globales o regionales. Esta dinámica evidencia la necesidad de contar con sistemas que analicen datos en tiempo real, integren fuentes heterogéneas y ofrezcan señales predictivas para anticipar cambios del mercado. Los sistemas multiagentes permiten distribuir tareas entre componentes especializados (por ejemplo, agentes de mercado, predicción, noticias o usuario), mejorando la velocidad y precisión de las decisiones.

En particular, la incorporación de modelos de *machine learning* para la clasificación de movimientos de precios y de modelos de lenguaje para el análisis de sentimiento en noticias financieras representa una tendencia consolidada en la literatura reciente [3, 6]. Sin embargo, la integración de ambos enfoques en una única plataforma accesible, modular y automatizada sigue siendo un desafío abierto, especialmente para usuarios minoristas o instituciones financieras de menor escala.

Como se muestra en la figura 1.1, el sistema transforma datos financieros en alertas personalizadas.

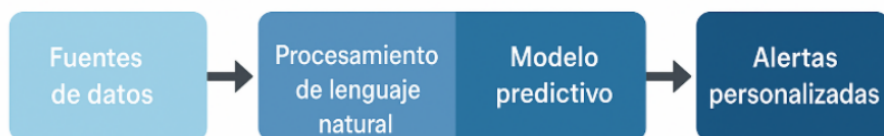


Figura 1.1. Flujo de información financiera.

1.2. Estado del arte

Diversas soluciones comerciales y académicas han abordado el desafío del análisis financiero automatizado mediante el uso de tecnologías avanzadas de IA y procesamiento de datos. Entre las plataformas comerciales más destacadas se encuentra Bloomberg Terminal, una herramienta ampliamente utilizada por instituciones financieras que centraliza información de mercado en tiempo real, ofrece indicadores económicos, métricas de riesgo y herramientas analíticas para la toma de decisiones estratégicas. Sin embargo, su costo elevado y su diseño orientado a usuarios expertos la convierten en una solución inaccesible para la mayoría de los inversionistas minoristas.

En el ámbito académico y tecnológico se han desarrollado modelos y arquitecturas específicas para el análisis de información financiera. En particular, los modelos de lenguaje entrenados en dominios financieros, como FinBERT, aplican técnicas de *Natural Language Processing* (NLP) para interpretar el sentimiento presente en noticias, reportes corporativos y publicaciones en redes sociales. Este tipo de análisis permite cuantificar el tono de la información (positivo, negativo o neutro) y utilizarlo como variable predictiva del comportamiento del mercado.

Diversos trabajos amplían el análisis de sentimiento en el ámbito financiero. Por ejemplo, Amola [7] propone un enfoque híbrido que integra el análisis de sentimiento de noticias con modelos cuantitativos tradicionales. De manera complementaria, estudios más recientes incorporan grandes modelos de lenguaje y mecanismos de recuperación de información contextual para mejorar la interpretación de textos financieros [8].

En paralelo, los modelos de series temporales como LSTM (*Long Short-Term Memory*) se utilizan para capturar dependencias temporales y estacionales en los datos y proyectar tendencias de precios, volatilidad y volumen de operaciones. Los modelos basados en LSTM han demostrado una alta capacidad para capturar dependencias temporales en series financieras [6, 9], e incluso se han combinado con análisis de sentimiento proveniente de redes sociales para mejorar la predicción de precios [10].

Más recientemente, los enfoques basados en *ensemble* de clasificadores han ganado relevancia en la predicción de la dirección de precios financieros. Métodos como Random Forest, Gradient Boosting y XGBoost han demostrado ser competitivos frente a modelos de mayor complejidad, especialmente cuando se combinan con un proceso riguroso de ingeniería de características técnicas y validación temporal [9]. Este tipo de enfoque, orientado a la clasificación binaria del movimiento del mercado (alza o baja), permite obtener señales accionables con mayor interpretabilidad que los modelos de regresión tradicionales.

No obstante, estas soluciones suelen presentarse de manera fragmentada, donde cada modelo o herramienta cumple un propósito aislado dentro del proceso

analítico. La integración de modelos predictivos, análisis de sentimiento y mecanismos automáticos de alerta en una única plataforma sigue siendo un desafío poco abordado. Además, la mayoría de estas implementaciones requieren infraestructura de cómputo de alto rendimiento, licencias costosas o conocimientos técnicos avanzados para su configuración y mantenimiento.

Como consecuencia, los inversionistas minoristas y entidades financieras regionales enfrentan una desventaja estructural frente a los grandes actores del mercado, al no disponer de soluciones tecnológicas asequibles que combinen accesibilidad, automatización y capacidad de adaptación. De allí surge la motivación para desarrollar un prototipo funcional de sistema inteligente multiagente, capaz de integrar estas funciones en una arquitectura modular y escalable, promoviendo así un acceso más equitativo al análisis financiero avanzado.

Como se muestra en la tabla 1.1, el prototipo propuesto presenta ventajas significativas en costo, accesibilidad e integración modular respecto de las soluciones existentes.

Tabla 1.1. Plataformas existentes vs. el prototipo propuesto.

Plataforma / Modelo	Costo de acceso	Accesibilidad	Personalización	Integración de módulos	Enfoque multi-agente	Tecnologías utilizadas
Bloomberg Terminal	Muy alto (licencia corporativa)	Limitada a instituciones financieras	Baja (interfaz cerrada)	Alta dentro de su ecosistema	No	API propietaria, bases de datos en tiempo real
FinBERT / Modelos NLP financieros	Medio (uso de modelos abiertos)	Media (requiere infraestructura técnica)	Alta (ajustable por dominio)	Media (depende de integración externa)	No	Python, PyTorch, Transformers
Modelos de series temporales (LSTM)	Bajo (software libre)	Alta (código abierto)	Alta (entrenable con datos propios)	Media (requiere ensamblaje manual)	No	Python, TensorFlow, PyTorch
Prototipo propuesto	Bajo / sin costo de licencia	Alta (interfaz web accesible)	Alta (configurable según usuario)	Alta (agentes interconectados)	Sí	Python, FastAPI, Streamlit, SQLite, ML + NLP

1.3. Motivación

La motivación central de este trabajo radica en la necesidad de reducir la brecha tecnológica y de información que separa a las grandes instituciones financieras de los pequeños inversionistas. Mientras los actores de gran escala cuentan con plataformas integradas, acceso prioritario a datos y capacidad de análisis automatizado, los usuarios minoristas suelen depender de información fragmentada, tardía o poco contextualizada y muchas veces carecen de conocimientos en el área de finanzas. Esta asimetría limita su capacidad para anticipar movimientos del mercado y tomar decisiones informadas.

El desarrollo de un sistema inteligente, autónomo y modular busca revertir esta situación mediante la provisión de alertas personalizadas y análisis automatizados, que permitan a cada usuario reaccionar de forma oportuna ante eventos financieros relevantes. La arquitectura propuesta integra componentes que combinan clasificación de dirección de precios mediante ensemble de modelos, análisis de sentimiento proveniente de noticias económicas y visualización de métricas, bajo un enfoque accesible, explicable y escalable.

El enfoque multiagente otorga al sistema flexibilidad y robustez, al permitir que cada agente cumpla un rol especializado y opere de manera cooperativa dentro de una estructura distribuida. Esta organización facilita la evolución, mantenimiento y ampliación del sistema, permitiendo incorporar nuevos módulos sin alterar su funcionamiento general.

La motivación de este trabajo se sustenta en el propósito de democratizar el acceso a la inteligencia financiera, brindando a los pequeños inversionistas una herramienta que les permita superar las barreras de conocimiento, tiempo y recursos tecnológicos, y fomentar así un ecosistema financiero más inclusivo, informado y equitativo.

1.4. Objetivos y alcance

A continuación, se desarrollan el objetivo general, los objetivos específicos y el alcance.

1.4.1. Objetivo general

Diseñar e implementar un sistema multiagente inteligente que integre análisis predictivo, procesamiento de noticias y generación de alertas personalizadas para el seguimiento de activos financieros.

1.4.2. Objetivos específicos

Los objetivos específicos son:

- Desarrollar agentes especializados para mercado, predicción, sentimiento, recomendación y alertas.
- Aplicar un ensemble de clasificadores de *machine learning* para predecir la dirección del precio de activos financieros.
- Implementar modelos NLP para análisis de sentimiento en noticias financieras.
- Diseñar una interfaz gráfica que permita visualizar métricas, predicciones y alertas.
- Evaluar el desempeño del sistema mediante métricas de clasificación.
- Implementar mecanismos de seguridad para garantizar la integridad de los datos, la confidencialidad de la información y la trazabilidad de las operaciones.

1.4.3. Alcance

El prototipo se limita al procesamiento de datos públicos y a la generación de alertas sobre un conjunto de activos seleccionados. No incluye operaciones reales de compra o venta, sino la demostración funcional del modelo multiagente y su capacidad de integración. La puesta en producción no forma parte del alcance de este trabajo; se propone como línea de trabajo futuro la integración del sistema en una aplicación web o móvil de uso institucional o minorista.

1.5. Impacto social y económico

La desigualdad informativa en los mercados financieros impacta directamente sobre la capacidad de decisión de los pequeños inversionistas. El sistema propuesto contribuye a democratizar el acceso a la información y la analítica avanzada, ofreciendo:

- Mayor transparencia en el seguimiento de activos financieros.
- Reducción de la brecha tecnológica entre grandes y pequeños actores del mercado.
- Potenciación de la educación financiera a través de herramientas accesibles y explicables.
- Fomento de la innovación tecnológica en el ámbito financiero regional.

Como se observa en la figura 1.2, el sistema reduce la brecha informativa entre actores financieros.



Figura 1.2. Brecha informativa entre grandes inversores y pequeños inversionistas.

Capítulo 2

Introducción específica

Este capítulo incluye los fundamentos teóricos y técnicos que sustentan el prototipo multiagente.

2.1. Sistema multiagente

Un sistema multiagente se basa en la interacción entre entidades de software denominadas agentes, que cooperan, compiten o se comunican entre sí para resolver tareas complejas [11].

En un entorno multiagente, el conocimiento y las responsabilidades están distribuidos, lo que permite una mayor flexibilidad, escalabilidad y tolerancia a fallos.

Un agente es una entidad autónoma capaz de percibir su entorno y actuar sobre él de manera racional [12].

Cada agente posee mecanismos internos para interpretar datos, planificar acciones y aprender de la experiencia, lo que le permite adaptarse dinámicamente a los cambios del contexto.

El ciclo básico de funcionamiento de un agente se compone de tres etapas principales:

- Percepción: recibe información de su entorno o de otros agentes; por ejemplo, precios de mercado, indicadores financieros o noticias relevantes.
- Decisión: analiza la información recibida, compara con sus objetivos y selecciona la acción más adecuada.
- Acción: ejecuta una tarea concreta o comunica los resultados a otros agentes, generando así un flujo continuo de información.

Estas etapas constituyen el ciclo percepción–decisión–acción, que caracteriza el comportamiento inteligente del sistema.

Los agentes presentan propiedades fundamentales que los distinguen de los programas convencionales:

- Autonomía: pueden actuar y tomar decisiones sin intervención directa del usuario o de un controlador central.
- Cooperación: colaboran con otros agentes mediante el intercambio de mensajes o eventos para alcanzar metas comunes.

- Modularidad: cada agente cumple un rol específico dentro del sistema, lo que permite dividir el trabajo en componentes especializados.
- Escalabilidad: el sistema admite la incorporación de nuevos agentes sin necesidad de modificar la estructura general.

En el prototipo desarrollado, cada agente cumple una función específica dentro del ecosistema:

- Agente de mercado
- Agente de predicción
- Agente de sentimiento
- Agente de recomendación
- Agente de alertas

La arquitectura propuesta se basa en un modelo distribuido de cooperación entre módulos. Como se muestra en la figura 2.1, cada agente es invocado secuencialmente por el backend FastAPI, que coordina el pipeline de procesamiento y centraliza las respuestas.

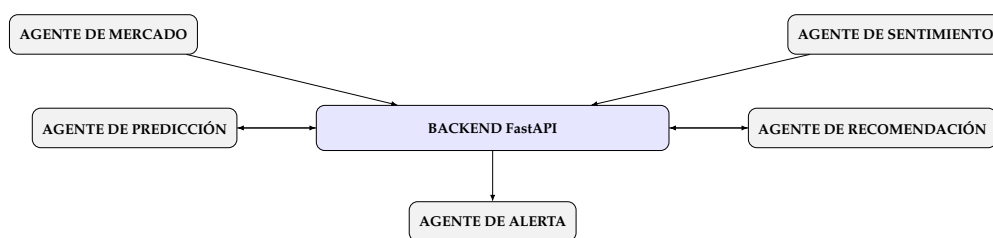


Figura 2.1. Interacción entre agentes en el sistema multiagente.

2.2. Machine learning aplicado a finanzas

El aprendizaje automático (*machine learning*) ha demostrado gran potencial para modelar y predecir el comportamiento de los mercados financieros. Los modelos de clasificación permiten capturar tendencias, estacionalidades y relaciones no lineales presentes en los precios de los activos.

Entre los algoritmos más utilizados para la clasificación de movimientos de precios se destacan los métodos de *ensemble*: Random Forest, Gradient Boosting, XGBoost y LightGBM. Estos clasificadores combinan múltiples modelos para mejorar la precisión y reducir el sobreajuste, y han demostrado ser competitivos en tareas de predicción financiera [9]. El proceso de entrenamiento se realiza mediante *walk-forward validation* con *TimeSeriesSplit*, respetando el orden temporal de los datos para evitar filtraciones de información futura. Las redes LSTM, presentadas por Hochreiter y Schmidhuber, han probado ser eficaces para capturar dependencias de largo plazo en datos financieros [13], y se incorporan en el sistema como componente opcional del ensemble.

En el presente trabajo, estos modelos se aplican para clasificar la dirección del movimiento de precios de activos financieros (alza o baja) en un horizonte de tres días, a partir de 52 características técnicas derivadas de los precios históricos. El entrenamiento utiliza una ventana de 504 días (2 años) con validación cruzada temporal de 5 folds. El target de clasificación se define con un umbral de 0,5 %:

se etiqueta como SUBIDA únicamente si el precio aumenta más del 0,5 %, lo que permite filtrar movimientos de menor magnitud que representan ruido de mercado. Los resultados se evalúan con métricas de clasificación binaria: Accuracy, Precision, Recall, F1-Score y AUC-ROC.

2.3. Arquitectura general

La arquitectura del sistema multiagente propuesto se basa en un ecosistema modular que permite la comunicación fluida entre componentes especializados. Su diseño busca maximizar la escalabilidad, la mantenibilidad y la capacidad de integración futura con servicios externos, empleando tecnologías abiertas y ligeras.

Cada agente se implementa como un módulo independiente, invocado secuencialmente por el backend FastAPI, que actúa como coordinador central del pipeline de procesamiento. El sistema se compone de tres elementos tecnológicos principales: FastAPI, SQLite y Streamlit, cuyas funciones se describen a continuación.

- FastAPI: framework moderno y de alto rendimiento para el desarrollo de servicios REST en Python. Su uso permite manejar peticiones concurrentes mediante asincronía nativa, lo que mejora la velocidad y escalabilidad del sistema [14].
- SQLite: base de datos relacional embebida, ligera y de código abierto, utilizada para almacenar configuraciones, métricas e históricos de predicciones sin necesidad de un servidor adicional [15].
- Streamlit: biblioteca orientada a la construcción rápida de interfaces gráficas y dashboards interactivos. Permite que el usuario final visualice métricas, alertas y resultados, integrando datos provenientes de los agentes [16].

2.3.1. Diagrama en bloques del sistema

La figura 2.2 presenta la arquitectura general del sistema multiagente en la que se destacan los principales módulos y el flujo de información entre ellos.

Como se mencionó anteriormente, el diseño adopta un enfoque modular y secuencial: los agentes son invocados por el backend FastAPI, que actúa como núcleo coordinador del pipeline de procesamiento y centraliza las respuestas del sistema. La base de datos SQLite almacena configuraciones, métricas e históricos de predicción, y la interfaz Streamlit ofrece una visualización interactiva para el usuario final.

Esta arquitectura modular garantiza la escalabilidad, la mantenibilidad y la posibilidad de incorporar nuevos componentes sin alterar el funcionamiento general del sistema.

El agente de predicción integra la información proveniente del mercado y del análisis de sentimiento, mientras que el agente de recomendación combina señales predictivas y de sentimiento para generar recomendaciones. El agente de alertas evalúa los resultados y determina el nivel de severidad correspondiente. Los módulos se comunican secuencialmente mediante FastAPI, y Streamlit constituye la capa de visualización e interacción con el usuario.

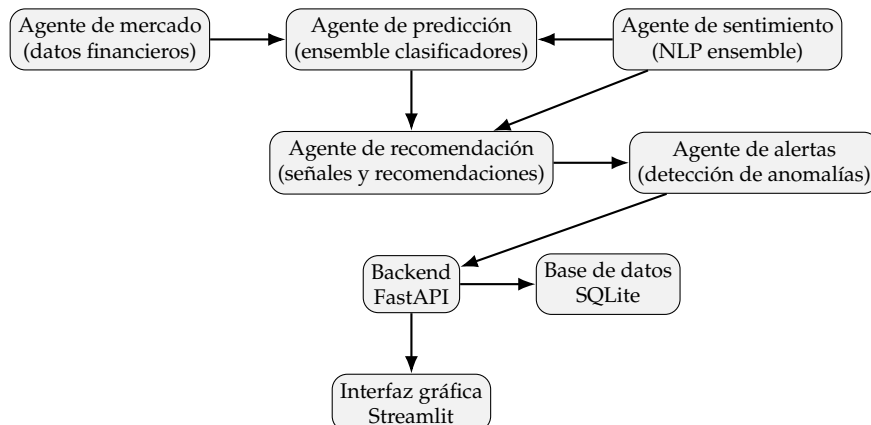


Figura 2.2. Arquitectura general del sistema multiagente.

2.3.2. Pipeline secuencial y modularidad

El sistema opera bajo un paradigma de pipeline secuencial, en el que cada agente es invocado en orden por el backend FastAPI: primero el agente de mercado, luego el de predicción, sentimiento, recomendación y finalmente el de alertas. Este enfoque simplifica la depuración, garantiza la trazabilidad del flujo de datos y es suficiente para el prototipo desarrollado. La coordinación centralizada en FastAPI permite además gestionar los tiempos de respuesta y manejar errores de forma uniforme.

La modularidad de la arquitectura permite reemplazar componentes (por ejemplo, migrar de SQLite a PostgreSQL o sustituir FinBERT por otro modelo NLP) sin afectar el funcionamiento del resto del sistema. De este modo, el prototipo combina simplicidad operativa con la posibilidad de escalar hacia entornos de producción más complejos.

2.4. Evaluación de riesgos y explicabilidad

A continuación se describen los riesgos en las predicciones financieras, el concepto de explicabilidad, riesgos operacionales y normativas de cumplimiento.

2.4.1. Riesgos en predicciones financieras

Los sistemas de IA aplicados a entornos financieros presentan riesgos técnicos, éticos y operativos que pueden afectar la confiabilidad del modelo y la calidad de las decisiones basadas en sus resultados. Entre los principales se destacan:

- **Sesgos en los datos.** Los modelos entrenados con información histórica tienden a reproducir patrones y distorsiones del pasado, lo que puede generar predicciones sesgadas por factores macroeconómicos, geográficos o regulatorios [17]. Un conjunto de datos poco representativo puede inducir resultados injustos o incorrectos.
- **Sobreajuste (overfitting).** Se produce cuando el modelo se ajusta excesivamente a los datos de entrenamiento y pierde capacidad de generalización [18]. En el sistema desarrollado, este riesgo se mitiga mediante *walk-forward validation* con *TimeSeriesSplit*, que respeta el orden temporal de los datos durante la evaluación.

- Dependencia excesiva de la automatización. La confianza ciega en modelos automatizados sin supervisión humana puede derivar en decisiones inadecuadas ante eventos atípicos o “cisnes negros” [19]. La intervención humana continúa siendo necesaria para contextualizar los resultados.
- Riesgos de interpretación y comunicación. En modelos de *ensemble*, las relaciones entre variables no siempre son evidentes. Una comunicación deficiente de los resultados puede generar errores de juicio o problemas de trazabilidad en auditorías.
- Riesgos de ciberseguridad y manipulación. Los sistemas que consumen datos desde APIs o portales de noticias pueden ser vulnerables a contenido manipulado, lo que afecta la integridad de las predicciones.

2.4.2. Explicabilidad

La explicabilidad (XAI, *Explainable Artificial Intelligence*) busca hacer comprensible el proceso mediante el cual un modelo llega a una predicción o recomendación [20]. En contextos financieros, la interpretabilidad es además una exigencia regulatoria vinculada a la transparencia, la rendición de cuentas y la protección de los usuarios.

En este trabajo se incorporan diferentes enfoques de explicabilidad:

- Importancia de características. Los modelos del ensemble (Random Forest, Gradient Boosting, XGBoost y LightGBM) exponen la contribución relativa de cada variable técnica a la predicción mediante `feature_importances_`. Esto posibilita identificar qué indicadores (como volatilidad, RSI o MACD) tuvieron mayor peso en una decisión concreta.
- Análisis de sensibilidad y correlación. Estos métodos permiten evaluar la estabilidad del modelo ante variaciones en sus variables de entrada y detectar relaciones entre indicadores financieros.
- Explicabilidad local y global. Mientras la explicabilidad local justifica decisiones puntuales (por ejemplo, una alerta emitida un día específico), la explicabilidad global caracteriza el comportamiento general del modelo. Ambas perspectivas son necesarias para una gobernanza integral del sistema.
- Trazabilidad y auditoría. Cada predicción y alerta se almacena en la base de datos junto con sus parámetros asociados, lo que permite reconstruir el proceso decisorio, una práctica fundamental en entornos regulados [21].

2.4.3. Riesgo operacional y robustez del modelo

Los modelos financieros pueden fallar en condiciones fuera de distribución, como crisis económicas o shocks inesperados. Danielsson y Macrae [22] subrayan que los algoritmos pueden degradarse abruptamente bajo estrés. La integración del análisis de sentimiento y la explicabilidad mediante importancia de características actúa como un mecanismo de alerta ante comportamientos inestables del modelo.

2.4.4. Cumplimiento regulatorio: Basilea III y BCRA comunicación A 7724

El sistema debe alinearse con los estándares regulatorios locales e internacionales. En particular:

- El marco de Basilea III establece lineamientos para la gestión del riesgo operativo y el uso controlado de modelos en los que destacan validación continua, documentación y gobierno del ciclo de vida [23].
- La Comunicación A 7724 del BCRA [24] define requisitos mínimos de gestión de riesgos informáticos, entre ellos:
 - Validación periódica del rendimiento del modelo.
 - Registro de parámetros y control de versiones.
 - Evaluación del impacto de decisiones automatizadas.
 - Mitigación del sobreajuste y la degradación temporal.

El sistema multiagente desarrollado incorpora mecanismos que se ajustan a las exigencias de Basilea III y con los requisitos de la Comunicación A 7724 del BCRA.

Capítulo 3

Diseño e implementación

El sistema implementa una arquitectura multiagente [25] que integra datos de mercado, predicción, análisis de sentimiento y generación de alertas inteligentes. Combina un backend desarrollado en FastAPI [26] con una interfaz en Streamlit y una base de datos SQLite orientada a trazabilidad y auditoría, en línea con las exigencias regulatorias actuales [24]. El pipeline procesa información desde la ingesta hasta la visualización en tiempo real, incorporando técnicas de explicabilidad (XAI) para mejorar la transparencia del sistema [20]. La seguridad se sustenta en JWT, rate limiting y registro de eventos conforme a las prácticas recomendadas por OWASP [27].

3.1. Arquitectura multiagente

La arquitectura propuesta se organiza alrededor de un conjunto de agentes especializados que cooperan entre sí para cubrir el ciclo completo de obtención de datos, análisis, generación de recomendaciones y emisión de alertas. Se destacan la autonomía, la cooperación y la modularidad como pilares para construir sistemas complejos, distribuidos y robustos.

Cada agente posee un objetivo bien definido, gestiona su propio estado interno y expone una interfaz clara para interactuar con el resto del sistema. La coordinación no se realiza de forma acoplada punto a punto, sino a través de un backend común que actúa como orquestador y de una base de datos compartida que funciona como memoria persistente. Esta organización facilita la escalabilidad horizontal (por ejemplo, replicando agentes de mercado para distintos activos) y reduce el riesgo de dependencias rígidas entre componentes.

En la versión actual del prototipo se han implementado cinco agentes funcionales:

- Agente de mercado.
- Agente de modelo o predicción.
- Agente de noticias y sentimiento.
- Agente de recomendación.
- Agente de alertas.

El agente de mercado se encarga de la obtención y preparación de datos numéricos. Para cada activo o ticker, descarga series de precios mediante APIs públicas (por ejemplo, servicios similares a Yahoo Finance), selecciona las columnas relevantes y aplica un preprocesamiento básico. Entre las tareas que realiza se

encuentran la limpieza de valores faltantes, la normalización de nombres de columnas, la verificación de orden temporal y el cálculo de indicadores agregados simples, como medias móviles. El resultado de este agente es un conjunto de datos coherente que puede ser consumido por los modelos de predicción sin necesidad de reimplementar lógica de limpieza en cada componente.

El agente de modelo o predicción toma las series de precios generadas por el agente de mercado y ejecuta algoritmos de aprendizaje automático sobre una ventana reciente de datos. El agente de modelo ejecuta un ensemble de cuatro clasificadores: Random Forest, Gradient Boosting, XGBoost y LightGBM, sobre una ventana de 504 días de datos históricos. El objetivo es predecir la dirección del precio del activo en un horizonte de tres días hábiles (clasificación binaria: alza o baja). El agente ajusta los modelos mediante validación cruzada temporal con cinco particiones y genera la variación porcentual esperada junto con las métricas de clasificación: accuracy, precisión, recall, F1-Score y AUC-ROC.

El agente de noticias y sentimiento analiza noticias financieras recientes del activo mediante un ensemble híbrido de cuatro componentes con pesos diferenciados: FinBERT (40 %), VADER (25 %), un lexicón financiero especializado (20 %) y TextBlob (15 %). El sistema obtiene las noticias directamente de Yahoo Finance y calcula un *score* de sentimiento en el intervalo $[-1, 1]$, junto con una etiqueta categórica (positivo, neutral o negativo) y un nivel de confianza.

El agente de recomendación combina la información obtenida de los agentes anteriores para producir una sugerencia textual interpretativa. A partir de la señal del mercado (por ejemplo, comparación entre precio actual y media móvil), la predicción cuantitativa (cambio porcentual esperado) y el sentimiento asociado, el agente construye una recomendación simplificada del tipo “revisar oportunidad de compra”, “considerar reducción de posición” o “mantener y monitorear”. La lógica se basa en reglas heurísticas transparentes, lo que contribuye a la explicabilidad del sistema y facilita su validación por parte de expertos humanos.

Finalmente, el agente de alertas actúa como mecanismo de vigilancia. Recibe como entrada la señal de mercado, la magnitud del cambio porcentual previsto y la recomendación generada. A partir de dichos insumos decide si corresponde registrar una alerta y con qué nivel de severidad. El criterio actual se basa en umbrales sobre el cambio porcentual esperado (por ejemplo, advertencias para variaciones mayores al 3 % y alertas críticas para variaciones mayores al 7 %). En caso de que se genere una alerta, el agente la persiste en la base de datos con su información asociada y queda así disponible tanto para visualización en el dashboard como para auditorías posteriores.

Desde el punto de vista del detalle técnico, cada agente opera sobre un volumen de datos y un conjunto de herramientas bien delimitados. El agente de mercado calcula 52 indicadores técnicos distribuidos en cuatro categorías (tendencia, momento, volatilidad y volumen), entre los que se incluyen SMA, EMA, MACD, RSI, Bandas de Bollinger, ATR, OBV y VWAP. El agente de modelo entrena un ensemble de cuatro clasificadores sobre una ventana de 504 días históricos con validación temporal de cinco particiones y garantiza que el conjunto de entrenamiento siempre preceda temporalmente al de validación. En cuanto al mecanismo de coordinación, el backend actúa como orquestador centralizado: ante cada solicitud, invoca los agentes en secuencia estricta, transfiere la salida de uno como entrada del siguiente y construye una respuesta JSON consolidada

que integra predicción, sentimiento, recomendación y alerta en un único objeto. Esta coordinación centralizada, a diferencia de un enfoque punto a punto entre agentes, reduce el acoplamiento y facilita la incorporación de nuevos agentes sin modificar los existentes.

La tabla 3.1 resume el alcance específico de cada agente, incluyendo sus funciones principales y las salidas que producen.

Tabla 3.1. Funciones y salidas de los agentes del sistema.

Agente	Función	Salida
Mercado	Obtiene series de precios y las prepara (limpieza, normalización, indicadores básicos).	Datos de mercado listos para modelado.
Modelo	Entrena un modelo y proyecta el valor futuro del activo.	Predicción y variación porcentual esperada.
Sentimiento	Analiza fuentes textuales para identificar polaridad y tono general.	Indicador de sentimiento y nivel de confianza.
Recomendación	Integra señales numéricas y cualitativas para generar una sugerencia.	Recomendación textual comprensible para el usuario.
Alertas	Evalúa umbrales y determina si corresponde registrar un evento crítico.	Alerta persistida con severidad y mensaje descriptivo.

La arquitectura incorpora además mecanismos de tolerancia a fallos: si el agente de modelo no puede generar una predicción, el sistema retorna el último precio conocido como valor de referencia; si el agente de sentimiento falla, se asigna automáticamente la etiqueta `NEUTRAL` con confianza cero, evitando que un fallo parcial interrumpa el pipeline completo.

La elección de una arquitectura multiagente frente a un pipeline monolítico responde a criterios de modularidad y escalabilidad: cada agente puede ser reemplazado, extendido o replicado de forma independiente sin afectar la lógica del resto del sistema.

La figura 3.1 representa de forma gráfica el flujo completo de interacción entre los agentes que componen la arquitectura del sistema. En ella se observa cómo los datos de mercado alimentan primero al agente de mercado y luego al agente de modelo, cuya salida se combina con el análisis cualitativo realizado por el agente de sentimiento. A partir de estas señales integradas, el agente de recomendación genera una sugerencia para el usuario, mientras que el agente de alertas evalúa la severidad del escenario y decide si corresponde registrar un evento crítico. Este recorrido refleja la secuencia lógica del pipeline y expone de manera clara la relación jerárquica y funcional entre los módulos.

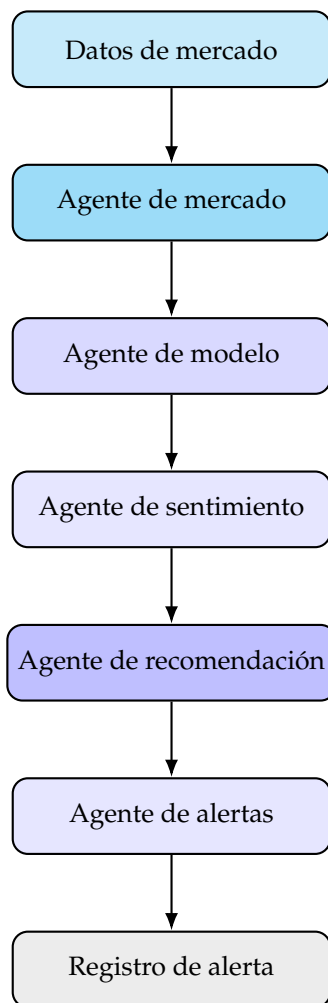


Figura 3.1. Diagrama vertical de interconexión entre agentes del sistema multiagente.

3.2. Backend basado en FastAPI

El backend del sistema se desarrolló utilizando FastAPI, un framework moderno para la construcción de APIs en Python, diseñado con foco en rendimiento, tipado estático y validación robusta de datos. FastAPI se integra de manera natural con la versión actual de Pydantic y ofrece una sintaxis declarativa para definir modelos de entrada y salida, así como para documentar automáticamente la API mediante OpenAPI.

La aplicación expone un conjunto de endpoints agrupados en routers. Entre los más relevantes se encuentran los dedicados a autenticación y gestión de usuarios, los orientados a la obtención de predicciones y los asociados al listado de alertas. El endpoint raíz proporciona un estado básico de la aplicación y permite verificar rápidamente que el servicio se encuentra en ejecución.

La gestión de usuarios se implementa en el router de autenticación, que expone los siguientes endpoints: `/auth/register` para la creación de nuevos usuarios, `/auth/login` para la obtención del token JWT, `/auth/me` para consultar el perfil del usuario autenticado, `/auth/forgot-password` y `/auth/reset-password` para el flujo de recuperación de contraseña, y `/auth/change-password`

para el cambio de contraseña con sesión activa. El endpoint de login recibe credenciales, verifica la combinación de usuario y contraseña contra la base de datos y, en caso de ser válida, devuelve un token de acceso firmado con JWT.

El router de predicción implementa el endpoint `GET /predict/{ticker}`, que constituye el punto de entrada principal del flujo multiagente. Cuando se invoca con un ticker determinado, el backend actúa como orquestador: delega en el agente de mercado la obtención de datos históricos, pasa esa información al agente de modelo para generar una predicción, solicita un análisis de sentimiento al agente correspondiente, consulta al agente de recomendación y, finalmente, invoca al agente de alertas, que decide si debe registrarse una nueva alerta en la base de datos. Toda esta secuencia se encapsula en una respuesta JSON que incluye el detalle de cada componente.

El router de alertas proporciona el endpoint `GET /alerts`, que permite listar las alertas almacenadas en la base de datos con soporte para paginación, filtrado por ticker y filtrado por estado de lectura. Adicionalmente, expone endpoints para marcar alertas como leídas y para consultar estadísticas.

Para facilitar la integración con aplicaciones cliente, FastAPI genera de forma automática una documentación interactiva accesible en `/docs`, basada en OpenAPI, que resulta especialmente útil para pruebas manuales y para que otros equipos técnicos puedan integrar el sistema sin necesidad de revisar en detalle la implementación interna.

El flujo completo que sigue una petición desde el cliente hacia el backend y luego hacia los agentes se resume en la figura 3.2, en la que se muestra la secuencia de pasos desde la solicitud inicial hasta la generación de la respuesta final.

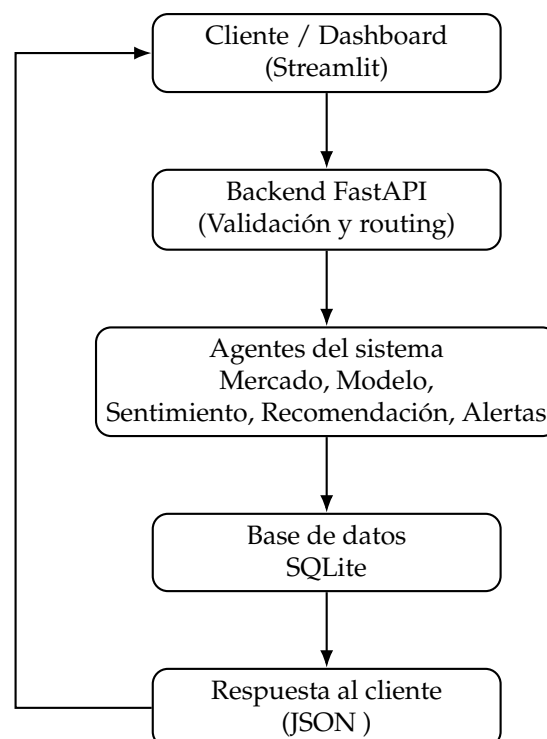


Figura 3.2. Diagrama vertical del flujo de peticiones API.

3.3. Interfaz en Streamlit

La interfaz de usuario se implementó utilizando Streamlit, un framework orientado a la construcción rápida de aplicaciones web de análisis de datos. Streamlit permite definir la estructura del dashboard directamente en Python, lo que simplifica el ciclo de desarrollo al evitar la necesidad de trabajar con tecnologías front-end tradicionales como HTML, CSS o JavaScript.

El dashboard propuesto se organiza en dos secciones principales, accesibles mediante pestañas: una dedicada a la predicción y señales, y otra enfocada en el histórico de alertas. El formulario de autenticación se presenta en la pantalla principal, donde el usuario puede iniciar sesión o registrarse. Al presionar el botón de inicio de sesión, la aplicación se comunica con el backend, invocando el endpoint de login. Si las credenciales son válidas, se almacena el token de acceso en la sesión de Streamlit, permitiendo que las solicitudes posteriores al backend incluyan automáticamente la cabecera de autorización requerida.

En la pestaña de predicción, el usuario puede ingresar el símbolo del activo que desea analizar. Al ejecutar la consulta, el dashboard envía una solicitud al endpoint de predicción del backend y, una vez recibido el resultado, presenta la información de forma organizada. Se muestran, por ejemplo, los valores de cierre recientes, la predicción para el siguiente período, el cambio porcentual esperado, la señal de mercado asociada y el sentimiento retornado por el agente correspondiente. La recomendación generada se representa como texto destacado, lo que ayuda a que el usuario pueda interpretar rápidamente el estado del activo.

En la pestaña de alertas se presenta una tabla con el histórico de alertas registradas. Para cada alerta se visualizan campos como el nivel de severidad, el ticker afectado, el mensaje descriptivo, la variación porcentual, los precios actual y predicho, el estado de lectura y la fecha de creación. Esta funcionalidad cumple dos roles: por un lado sirve como panel de control para el usuario final y, por otro, funciona como evidencia de trazabilidad, ya que permite reconstruir qué eventos fueron considerados suficientemente relevantes como para disparar una alerta.

Aunque el prototipo no incorpora gráficos complejos, la arquitectura de Streamlit permite añadir fácilmente visualizaciones adicionales, tales como series temporales de precios, comparaciones entre predicciones y valores reales, o distribuciones de alertas por nivel de severidad. Estas extensiones contribuirían a reforzar el carácter explicativo del sistema y facilitarían la exploración de su comportamiento en distintos escenarios.

La figura 3.3 ilustra la pantalla principal del dashboard en su estado inicial, donde el usuario puede autenticarse o crear una nueva cuenta antes de acceder a las funcionalidades del sistema.

Sistema de seguimiento y alertas para activos financieros

Plataforma de análisis para inversores minoristas

The image shows a web interface for a financial asset tracking and alert system. It features two main sections: 'Iniciar Sesión' (Login) and 'Crear Cuenta' (Create Account). The login section includes input fields for 'Usuario' (Username) and 'Contraseña' (Password), an 'Ingresar' (Login) button, and a link for '¿Olvidaste tu contraseña?' (Forgot your password?). The create account section includes input fields for 'Usuario', 'Email', 'Contraseña', and 'Confirmar Contraseña', and a 'Registrarse' (Register) button. The interface is clean and modern, with a light gray background and white form boxes.

Figura 3.3. Pantalla principal del dashboard Streamlit: formulario de autenticación e inicio de sesión.

La comunicación entre el dashboard y el backend se articula en torno a tres endpoints principales, cuyo flujo se esquematiza en la figura 3.4. El proceso inicia con el formulario de login, que envía las credenciales al endpoint `POST /auth/login` y almacena el token JWT resultante en la sesión de Streamlit. Las solicitudes de predicción se canalizan a `GET /predict/{ticker}`, autenticadas mediante ese token, y retornan el análisis completo del activo. El histórico de alertas se recupera desde `GET /alerts`, también con autenticación Bearer.

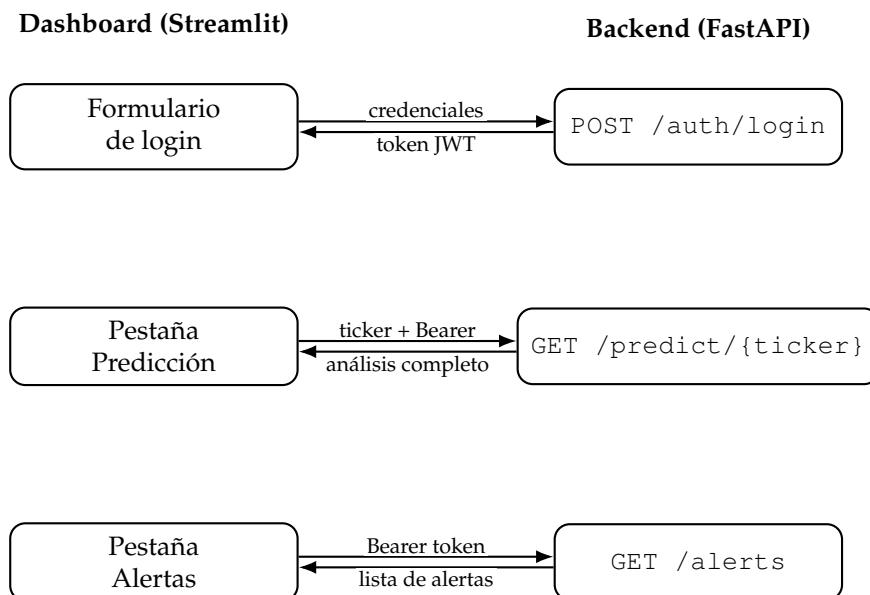


Figura 3.4. Flujo de comunicación entre el dashboard Streamlit y los tres endpoints principales del backend.

3.4. Base de datos

La solución utiliza una base de datos relacional ligera, basada en SQLite, adecuada para proyectos de prototipo y entornos de laboratorio.

La elección de SQLite responde a consideraciones de simplicidad de despliegue, portabilidad y ausencia de requerimientos de infraestructura adicional, sin dejar de cumplir con los requisitos de persistencia, integridad referencial y trazabilidad necesarios para el análisis posterior.

El acceso a la base se realiza a través de SQLAlchemy, que permite definir los modelos de datos mediante clases de Python mapeadas a tablas mediante el patrón ORM.

Entre las tablas principales se encuentran las correspondientes a usuarios, alertas, métricas de modelos y tokens de recuperación de contraseña. Cada tabla fue diseñada para capturar únicamente la información necesaria para el funcionamiento del sistema y la posterior auditoría de sus decisiones.

La tabla 3.2 presenta la estructura de la tabla de usuarios, que almacena la identificación, el nombre de usuario, el correo electrónico (en caso de estar disponible), la contraseña cifrada, el rol asignado y la fecha de registro. Este diseño permite diferenciar entre administradores y analistas, lo que facilita la implementación de controles de acceso basados en roles en una etapa posterior.

Tabla 3.2. Estructura de la tabla usuarios.

Campo	Tipo	Descripción
id	INTEGER (PK)	Identificador único del usuario.
username	TEXT	Nombre de usuario para autenticación.
email	TEXT	Correo electrónico asociado (opcional).
password_hash	TEXT	Hash seguro de la contraseña (bcrypt).
rol	TEXT	Rol asignado (administrador o analista).
fecha_creacion	DATETIME	Fecha y hora de registro del usuario.

La tabla 3.3 resume la estructura de la tabla de alertas.

Tabla 3.3. Estructura de la tabla alertas.

Campo	Tipo	Descripción
id	INTEGER (PK)	Identificador interno de la alerta.
usuario_id	INTEGER (FK)	Usuario asociado a la alerta.
ticker	TEXT	Activo financiero que originó la alerta.
tipo	TEXT	Nivel de severidad
mensaje	TEXT	Descripción de la alerta generada.
variacion_pct	REAL	Variación porcentual esperada.
precio_actual	REAL	Precio de mercado actual
precio_predicho	REAL	Precio proyectado por el modelo.
leida	INTEGER	0 = no leída, 1 = leída.
fecha_creacion	DATETIME	Momento de creación de la alerta.

En esta tabla de alertas, se registra para cada evento el identificador interno, el usuario asociado, el ticker afectado, el tipo de severidad, un mensaje descriptivo, la variación porcentual prevista, los precios actual y proyectado, el estado de lectura y la marca de tiempo de creación. Estos datos posibilitan reconstruir, para cualquier activo y período, qué eventos fueron considerados relevantes por el sistema y con qué intensidad.

La tabla 3.4 muestra la estructura de la tabla de métricas de modelo, que registra el desempeño del clasificador binario en cada predicción. Dado que el modelo predice la dirección del precio (SUBIDA/BAJADA), las métricas almacenadas son propias de clasificación: exactitud general (Accuracy), precisión y recall sobre la clase positiva (SUBIDA), su balance armónico (F1-Score) y el área bajo la curva ROC (AUC-ROC). De esta manera, es posible monitorear la evolución temporal de la capacidad discriminativa del modelo e identificar posibles degradaciones debidas a cambios estructurales en el mercado.

Tabla 3.4. Estructura de la tabla `metricas_modelo`.

Campo	Tipo	Descripción
<code>id</code>	INTEGER (PK)	Identificador único del registro de métrica.
<code>usuario_id</code>	INTEGER (FK)	Usuario que ejecutó la predicción (opcional).
<code>ticker</code>	VARCHAR(20)	Símbolo del activo financiero analizado.
<code>modelo</code>	VARCHAR(50)	Nombre del modelo utilizado (Clasificador).
<code>accuracy</code>	REAL	Exactitud general del clasificador (Accuracy).
<code>precision</code>	REAL	Precisión de clase positiva, SUBIDA (Precision).
<code>recall</code>	REAL	Sensibilidad de clase positiva, SUBIDA (Recall).
<code>f1</code>	REAL	Balance entre precisión y recall (F1-Score).
<code>auc</code>	REAL	Área bajo la curva ROC (AUC-ROC).
<code>fecha</code>	DATETIME	Fecha y hora del registro.

El esquema incluye además una tabla auxiliar `password_reset_tokens`, que almacena los tokens temporales generados durante el flujo de recuperación de contraseña. Cada token está vinculado a un usuario mediante clave foránea, registra su fecha de expiración y un indicador de uso, garantizando que cada token pueda emplearse una única vez.

La figura 3.5 sintetiza la estructura global de la base de datos mediante un diagrama entidad-relación que ilustra, de forma directa, cómo interactúan los elementos centrales del sistema. En este modelo, la entidad usuarios se vincula con la entidad métricas mediante una relación 1:N, lo que indica que cada usuario puede poseer múltiples registros de desempeño del modelo. A su vez, métricas se relaciona con alertas también mediante una relación 1:N, reflejando que cada evaluación del modelo puede dar origen a una o varias alertas registradas.

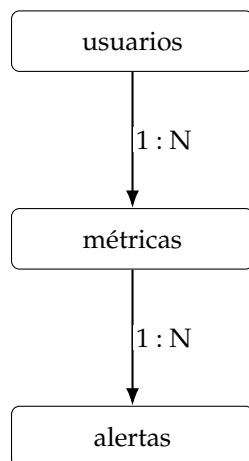


Figura 3.5. Diagrama entidad-relación de la base de datos.

Este encadenamiento estructural permite reconstruir, para cualquier alerta emitida, qué métrica del modelo la generó y a qué usuario pertenece dicha métrica, lo que garantiza la trazabilidad completa para auditorías, análisis posteriores y explicabilidad. El esquema ER, complementado con las tablas detalladas de campos, tipos de datos y claves primarias/foráneas, constituye así la documentación técnica fundamental para la evolución futura del sistema y permite extender la base de datos (o integrarla con otros módulos) de manera controlada, consistente y verificable.

3.5. Sistema de alertas

El sistema de alertas constituye el componente más visible para el usuario cuando se superan ciertos umbrales de riesgo o se detectan escenarios potencialmente críticos. El flujo se inicia en el endpoint de predicción: una vez finalizado el cálculo, el backend evalúa, junto con el agente de alertas, si el cambio porcentual previsto para el activo y la señal de mercado justifican la creación de una notificación.

El sistema integra un canal de notificación que es la consulta REST al endpoint `/alerts/`. A través de este canal, el dashboard recupera y presenta las alertas persistidas en la base de datos. La integración de notificaciones push mediante WebSocket y el envío de correo electrónico ante alertas críticas quedan propuestos como trabajo futuro.

El agente de alertas evalúa de forma continua los resultados de predicción y los indicadores de sentimiento. Cuando la magnitud del cambio porcentual esperado supera umbrales previamente definidos, se genera una advertencia o una alerta crítica, según la severidad del movimiento.

La figura 3.6 resume este proceso desde la obtención del resultado de predicción hasta el envío de la notificación al usuario para las acciones de APPLE.

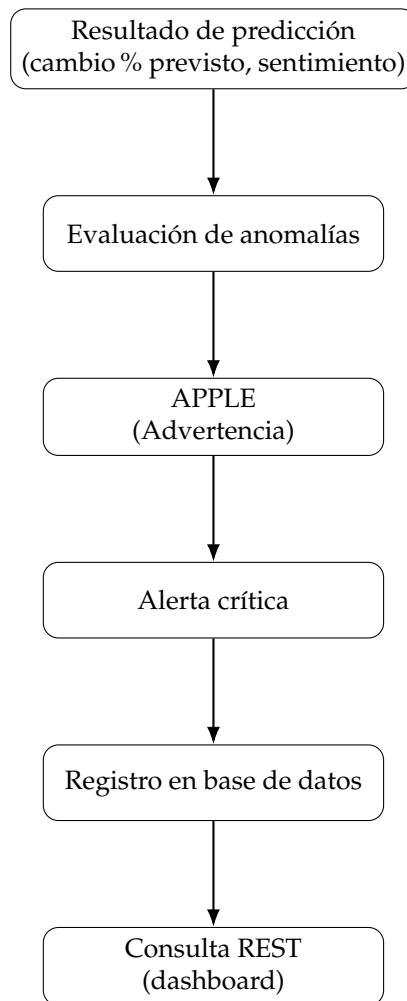


Figura 3.6. Diagrama vertical del flujo de generación y consulta de alertas.

En el prototipo actual, el criterio de decisión combina múltiples técnicas de detección de anomalías: Z-Score, MAD (*Median Absolute Deviation*), CUSUM para cambios de tendencia, EWMA para control estadístico e Isolation Forest para anomalías multivariadas. El resultado integrado de estos detectores determina uno de seis niveles de severidad: INFO, LOW, MEDIUM, HIGH, CRITICAL y EMERGENCY.

El sistema distingue además nueve tipos de alerta: movimiento de precio, pico de volatilidad, cambio de tendencia, anomalía detectada, volumen inusual, cambio de sentimiento, divergencia de predicción, ruptura de correlación y patrón detectado. Estos parámetros pueden ajustarse según el perfil de riesgo del usuario y la volatilidad típica de los activos bajo análisis.

Una vez tomada la decisión, el agente de alertas construye un registro con los campos definidos en la base de datos (usuario, activo, tipo, mensaje, variación prevista, precio actual, precio predicho y fecha) y lo persiste en la tabla correspondiente.

La visualización de las alertas en la interfaz se realiza a través del endpoint de listado, que recupera los registros más recientes y los presenta en forma de histórico filtrable.

La tabla 3.5 muestra un ejemplo simplificado de las alertas que pueden registrarse durante la operación del sistema para distintos activos financieros.

Tabla 3.5. Ejemplo de alertas generadas por movimientos fuera del rango esperado .

Fecha y hora	Ticker	Mensaje resumido	Var. (%)	Canal
2025-11-03 10:22	AAPL	Variación moderada fuera del rango esperado según el modelo (estado APPLE).	+2,8	Consulta REST
2025-11-03 12:40	TSLA	Incremento abrupto en la volatilidad intradía identificado por el sistema.	-3,4	Consulta REST
2025-11-03 15:10	NVDA	Desvío significativo respecto de la tendencia prevista por el modelo predictivo.	+3,1	Consulta REST

El sistema implementa un servicio SMTP con autenticación segura, en línea con las recomendaciones de seguridad propuestas por OWASP [27], utilizado para notificaciones de autenticación: recuperación de contraseña y confirmación de cambio de contraseña. El envío de correos ante alertas financieras críticas queda propuesto como trabajo futuro.

3.6. Seguridad

La arquitectura del sistema incorpora un conjunto de mecanismos de seguridad orientados a garantizar la integridad de los datos, la confidencialidad de la información sensible y la trazabilidad de las operaciones.

Estas medidas se encuentran alineadas tanto con las buenas prácticas internacionales propuestas por OWASP [27], como con los lineamientos establecidos por el Banco Central de la República Argentina para la gestión de riesgos informáticos [24].

Entre los mecanismos implementados se destacan:

- Autenticación robusta mediante *JSON Web Tokens* (JWT).
- Hash seguro de contraseñas utilizando `bcrypt`.
- Validación estricta de parámetros mediante modelos `Pydantic`.
- Rate limiting de alertas para evitar saturación de notificaciones repetidas por activo.
- Registro estructurado de eventos (logging con marca temporal, módulo y nivel de severidad).

En primera instancia, el sistema implementa un mecanismo de autenticación basado en JWT, en el que cada usuario recibe un token firmado digitalmente con una clave privada almacenada en variables de entorno. Este token contiene la identidad del usuario, información adicional para autorización y una fecha de expiración que limita su uso. La validación de los tokens se realiza en cada endpoint protegido, lo que reduce así el riesgo de accesos no autorizados.

Las contraseñas de los usuarios nunca se almacenan en texto plano. El backend utiliza la biblioteca `Passlib` con el algoritmo `bcrypt`, que aplica funciones de hash lentas y resistentes a ataques de fuerza bruta. En caso de comprometerse la base de datos, los hashes no pueden revertirse mediante técnicas de descifrado directo, cumpliendo con los estándares recomendados por OWASP para la protección de credenciales.

Otro de los pilares del sistema es la validación rigurosa de los datos de entrada. FastAPI, apoyado en Pydantic, garantiza que todo parámetro recibido cumple con el tipo, formato y restricciones definidas en los modelos. Esto evita procesar solicitudes incorrectas, incompletas o manipuladas con intención maliciosa, lo que reduce significativamente la superficie de ataque.

El sistema implementa rate limiting a nivel de generación de alertas, controlando la frecuencia con que se emiten notificaciones por activo financiero para evitar saturación de alertas repetidas. La limitación de frecuencia sobre los endpoints HTTP de la API queda propuesta como trabajo futuro.

La gestión de secretos del sistema se realiza mediante variables de entorno, evitando la inclusión de credenciales directamente en el código fuente. La clave secreta utilizada para firmar los tokens JWT, así como los parámetros de conexión a la base de datos, se cargan en tiempo de ejecución a través de *Pydantic Settings*, garantizando que información sensible no quede expuesta en repositorios de control de versiones ni en artefactos de despliegue.

Finalmente, el sistema incorpora un módulo de auditoría que registra eventos con marca temporal, módulo origen y nivel de severidad mediante el módulo estándar de logging de Python. Cada operación relevante queda trazada con estos parámetros descriptivos, lo que facilita la detección de anomalías, la investigación de incidentes y el cumplimiento de los requerimientos regulatorios del BCRA para sistemas tecnológicos en instituciones financieras.

La figura 3.7 sintetiza visualmente estas capas de protección, mostrando la secuencia de validaciones y controles que intervienen en cada solicitud. Mediante este diagrama, se explica cómo el objetivo específico de la seguridad se concreta en el presente trabajo.

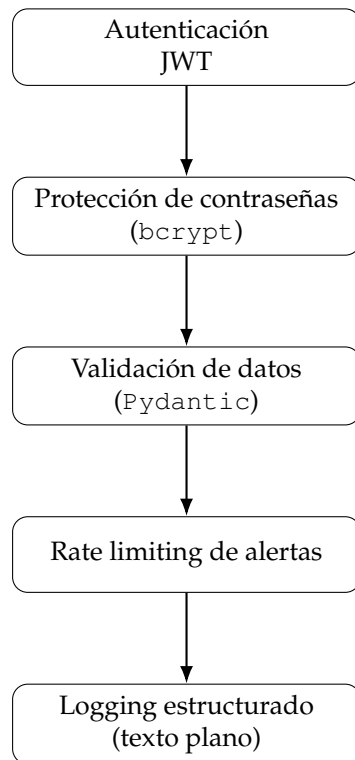


Figura 3.7. Diagrama del flujo de seguridad implementado en el sistema.

La tabla 3.6 resume las funciones de cada componente e ilustra cómo contribuyen al modelo integral de seguridad del sistema.

Tabla 3.6. Componentes de seguridad implementados.

Componente	Descripción
Autenticación JWT	Controla el acceso a endpoints protegidos mediante tokens firmados que incluyen identidad, permisos y expiración.
Hash <code>bcrypt</code>	Protege credenciales aplicando un hash lento y resistente a ataques de fuerza bruta, conforme a OWASP.
Validación de datos	Comprueba tipos, formatos y restricciones de todos los parámetros recibidos, evitando entradas malformadas o maliciosas.
Rate limiting de alertas	Controla la frecuencia con que se emiten notificaciones por activo financiero, evitando saturación de alertas repetidas.
Logging estructurado	Registra acciones relevantes con marca temporal, módulo origen y nivel de severidad para auditoría, detección de anomalías y cumplimiento normativo BCRA.

3.7. Pipeline de procesamiento

El pipeline de procesamiento constituye el eje central del funcionamiento del sistema, ya que define la secuencia lógica mediante la cual los datos brutos son transformados en información procesada, evaluada y finalmente presentada al usuario en forma de predicciones y alertas. Su diseño modular permite que cada etapa funcione de manera independiente y coordinada, favoreciendo la extensibilidad y la incorporación futura de nuevos modelos o fuentes de información.

El flujo completo está compuesto por siete etapas principales:

1. Ingesta y validación de datos de mercado.
2. Normalización y limpieza de series.
3. Ejecución de modelos predictivos.
4. Análisis de noticias y sentimiento.
5. Integración de señales y generación de alertas.
6. Almacenamiento estructurado en base de datos.
7. Visualización interactiva en el dashboard.

La modularidad del pipeline permite incorporar nuevos activos, modelos o fuentes de datos sin modificar la estructura base de la arquitectura, dado que cada etapa está encapsulada en un agente especializado y se interconecta mediante el backend.

En primer lugar, se encuentra la etapa de ingesta de datos, en la que el agente de mercado descarga las series históricas del activo seleccionado. Este proceso contempla la comunicación con proveedores externos, la captura de posibles errores de red, la verificación de integridad y la adaptación del formato recibido a la estructura interna del sistema. Solo si los datos superan estas verificaciones se habilita su uso en las etapas siguientes.

Luego se realiza el preprocesamiento y normalización, que incluye selección de columnas relevantes, tratamiento de valores faltantes, ajuste de escalas, conversión de tipos de datos y cálculo de indicadores agregados (por ejemplo, variación porcentual o medias móviles). Este paso resulta fundamental para garantizar que el modelo predictivo trabaje con un conjunto de datos limpio y coherente.

A continuación, en la etapa de modelado, el agente de predicción entrena un ensemble de clasificadores (Random Forest, Gradient Boosting, XGBoost y LightGBM) sobre una ventana móvil de observaciones recientes.

Se utiliza walk-forward validation temporal. Como resultado, el sistema genera una probabilidad de movimiento alcista o bajista para el activo en el horizonte de tres días, acompañada de métricas de clasificación (Accuracy, Precision, Recall, F1-Score y AUC-ROC).

Paralelamente, el agente de sentimiento consulta noticias recientes del activo mediante Yahoo Finance y las analiza con un ensemble de cuatro modelos: FinBERT (40 %), VADER (25 %), TextBlob (15 %) y un lexicón financiero propio de más de 500 términos (20 %). El resultado es una señal cualitativa que complementa la información cuantitativa del modelo predictivo.

La siguiente fase corresponde a la integración y recomendación. El agente de recomendación combina la predicción cuantitativa, la señal de mercado y la información de sentimiento para generar un mensaje comprensible para el usuario. Sobre esta base se ejecuta la evaluación de alertas, en la que el agente correspondiente determina el nivel de severidad (INFO, LOW, MEDIUM, HIGH, CRITICAL o EMERGENCY) mediante técnicas estadísticas de detección de anomalías, y persiste el registro en la base de datos para su consulta desde el dashboard.

Finalmente, los resultados se persisten en la base de datos junto con su marca temporal, lo que permite la auditoría y trazabilidad. La API devuelve la información en formato JSON, facilitando que el dashboard de Streamlit visualice predicciones, métricas y alertas.

La figura 3.8 muestra una representación vertical del pipeline, en la que se destaca el rol de cada agente y la secuencia de transformación de los datos hasta su presentación final al usuario.

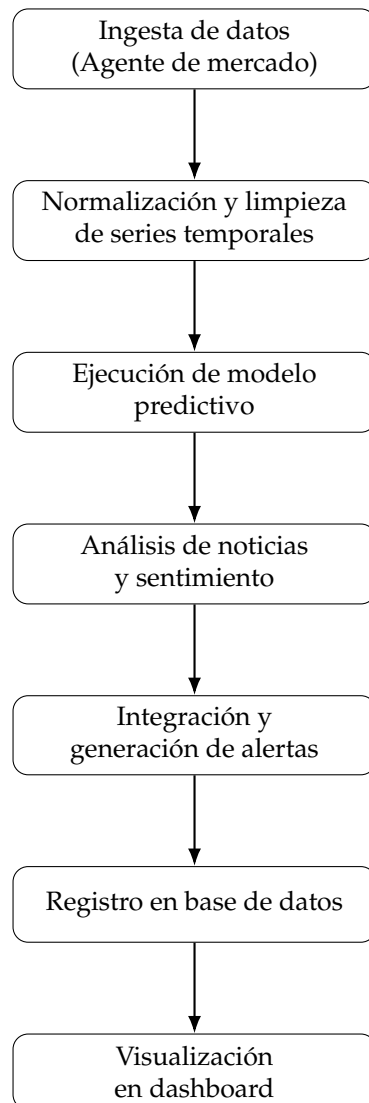


Figura 3.8. Diagrama vertical del pipeline completo de procesamiento, desde la ingesta hasta la visualización.

3.8. Despliegue y entorno

El sistema se despliega utilizando:

- Archivos `requirements.txt` para gestionar dependencias.
- Servidores ASGI (Uvicorn) para ejecución del backend.
- Variables de entorno para claves sensibles.

Esta estrategia garantiza que las pruebas locales sean reproducibles y fácilmente transportables entre equipos de trabajo.

El backend se distribuye junto con un archivo `requirements.txt` que declara las dependencias de Python necesarias para su ejecución, incluyendo `FastAPI`, `Uvicorn`, `SQLAlchemy`, `Pydantic`, las bibliotecas de seguridad y los paquetes para tratamiento de datos.

La configuración sensible, como claves secretas y cadenas de conexión a la base de datos, se gestiona a través de variables de entorno definidas en un archivo `.env`. La clase de configuración del backend, basada en `Pydantic Settings`, se encarga de cargar estos valores y ponerlos a disposición del resto de la aplicación sin que queden expuestos en el código fuente.

Para la ejecución local, el servidor se activa mediante Uvicorn, un servidor ASGI ligero. De esta manera, es posible probar la API en un entorno de desarrollo con una simple instrucción de consola. La misma configuración puede adaptarse a entornos de producción aumentando el número de trabajadores o integrando el servicio en un servidor web más robusto.

La estructura de carpetas y la separación entre backend y dashboard facilita la creación futura de imágenes Docker independientes (por ejemplo, una para la API y otra para la interfaz de usuario), lo que queda propuesto como trabajo futuro.

La tabla 3.7 resume las principales bibliotecas utilizadas, su propósito dentro del sistema y su tipo de funcionalidad.

Tabla 3.7. Bibliotecas principales.

Paquete	Categoría	Función dentro del sistema
<code>fastapi</code>	Backend API	Framework principal para definir endpoints, validar datos y generar documentación OpenAPI.
<code>uvicorn</code>	Servidor ASGI	Ejecuta la aplicación FastAPI y permite un manejo concurrente y despliegue liviano.
<code>pydantic</code>	Validación	Modelado estricto de datos y carga segura de variables de entorno mediante <code>pydantic-settings</code> .
<code>sqlalchemy</code>	Base de datos	ORM para definir modelos, manejar sesiones y ejecutar consultas sobre SQLite.

Continúa en la página siguiente

Tabla 3.7 – continuación

Paquete	Categoría	Función dentro del sistema
passlib	Seguridad	Hash seguro de contraseñas con <code>bcrypt</code> , resistente a ataques de fuerza bruta según OWASP.
python-jose	Seguridad	Generación y validación de tokens JWT para autenticación de usuarios.
yfinance	Datos financieros	Descarga de series históricas de precios y noticias recientes de activos desde Yahoo Finance.
pandas	Procesamiento	Limpieza, manipulación y normalización de series históricas de precios.
numpy	Cálculo numérico	Soporte matemático para operaciones vectorizadas y cálculo de indicadores técnicos.
ta	Análisis técnico	Cálculo de más de 20 indicadores técnicos (RSI, MACD, Bollinger Bands, ATR, OBV, entre otros).
scikit-learn	Modelado	Clasificadores base (Random Forest, Gradient Boosting), preprocesamiento y métricas de clasificación.
xgboost	Modelado	Clasificador XGBoost para predicción binaria de dirección de precios, parte del ensemble.
lightgbm	Modelado	Clasificador LightGBM para predicción binaria de dirección de precios, parte del ensemble.
torch	Deep Learning	Implementación opcional del modelo LSTM para predicción de series temporales.
transformers	NLP	Carga del modelo FinBERT (ProsusAI/finbert) para análisis de sentimiento financiero.
nltk	NLP	Análisis de sentimiento mediante VADER, optimizado para textos de redes sociales y noticias.
textblob	NLP	Análisis de polaridad general de texto, componente del ensemble de sentimiento.
streamlit	Dashboard	Interfaz de usuario interactiva para visualización de predicciones, métricas y alertas.
plotly	Visualización	Gráficos interactivos de precios históricos y predicciones en el dashboard.

Continúa en la página siguiente

Tabla 3.7 – continuación

Paquete	Categoría	Función dentro del sistema
python-dotenv	Configuración	Carga de variables de entorno desde el archivo <code>.env</code> sin exponer credenciales en el código.
smtpplib (estándar)	Notificaciones	Envío de correos de recuperación y confirmación de cambio de contraseña mediante SMTP autenticado.

3.9. Monitoreo y logging

El último componente del sistema es el módulo de monitoreo y logging, que tiene como finalidad registrar de manera sistemática los eventos relevantes y proporcionar información útil para la detección temprana de problemas.

El módulo registra:

- Errores y excepciones.
- Tiempos de respuesta por endpoint.
- Frecuencia y severidad de alertas.

El uso de logging estructurado facilita auditorías completas y permite detectar anomalías comportamentales en fases tempranas, reforzando la robustez operativa del sistema.

Cada vez que se ejecutan operaciones clave, como la emisión de una predicción o el listado de alertas, se genera un registro con marca de tiempo, nivel de severidad, módulo origen y un mensaje descriptivo. Estos registros se visualizan en la consola y se persisten en archivo mediante el módulo estándar de logging de python, con rotación diaria automática y retención de siete días, lo que garantiza trazabilidad sin consumo excesivo de almacenamiento. La redirección a sistemas centralizados de monitoreo queda reservada para versiones futuras.

El monitoreo del tiempo de respuesta de los endpoints, la frecuencia de llamadas a la API y la tasa de generación de alertas pueden incorporarse en futuras iteraciones del trabajo mediante la integración con soluciones como Prometheus o servicios equivalentes. La información recabada no solo es útil desde el punto de vista técnico, sino que también contribuye a demostrar el cumplimiento de buenas prácticas operativas y de requisitos regulatorios vinculados con el seguimiento de sistemas automatizados en el ámbito financiero.

En conjunto, la arquitectura multiagente, el backend, la interfaz de usuario, la base de datos, el sistema de alertas, los mecanismos de seguridad, el pipeline de procesamiento, el entorno de despliegue y las capacidades de monitoreo conforman una solución integrada que concreta, en un prototipo funcional, los objetivos planteados para el trabajo final.

Capítulo 4

Ensayos y resultados

El presente capítulo tiene como objetivo principal validar empíricamente la efectividad del sistema propuesto mediante una serie exhaustiva de experimentos, pruebas y análisis de casos reales. Asimismo, se presenta un análisis crítico que permite identificar tanto las fortalezas del sistema, tales como su robustez, consistencia e integración efectiva de múltiples agentes, como sus limitaciones, entre ellas la escalabilidad restringida a 10 usuarios concurrentes, la dependencia de servicios externos y la precisión variable según la volatilidad del activo. De este modo, se establecen las bases para las mejoras futuras que se discutirán en el Capítulo 5.

4.1. Validación integral del sistema propuesto

La validación de un sistema de inteligencia artificial en el dominio financiero requiere un enfoque riguroso y multidimensional. No basta con demostrar que los modelos de *Machine Learning* alcanzan métricas aceptables en conjuntos de validación; es necesario evaluar el sistema completo en condiciones operativas reales, considerando aspectos de rendimiento, escalabilidad, usabilidad y, fundamentalmente, el valor agregado que proporciona a usuarios finales con diferentes perfiles y objetivos de inversión.

Los resultados presentados en este capítulo fueron obtenidos en cuatro sesiones de prueba con datos reales de mercado obtenidos mediante la API de Yahoo Finance: una sesión de pruebas de carga y funcionales el 9 de febrero de 2026 (configuración inicial: ventana de entrenamiento de 30 días), una sesión de evaluación de modelos el 13 de febrero de 2026, una tercera sesión el 8 de marzo de 2026 en la que se aplicaron tres mejoras al ModelAgent (ampliación de la ventana de entrenamiento de 252 a 504 días, incremento de 3 a 5 folds en la validación cruzada temporal, e incorporación de un umbral del 0,5 % en la definición del target) y se re-evaluaron las métricas de clasificación, y una cuarta sesión el 21 de marzo de 2026.

Las métricas de la sección 4.2 corresponden a la configuración del 8 de marzo de 2026. Se siguió un protocolo de experimentación que incluye validación cruzada temporal para evitar data leakage y documentación de las condiciones experimentales para garantizar la reproducibilidad de los resultados.

4.2. Evaluación de modelos ML

Se presenta una evaluación detallada del ModelAgent, incluyendo la metodología de validación cruzada temporal, las métricas de clasificación obtenidas (Accuracy, Precision, Recall, F1-Score, AUC-ROC) y un análisis comparativo del rendimiento del ensemble de cuatro clasificadores (Random Forest, Gradient Boosting, XGBoost y LightGBM). Se tomaron diez acciones representativas de diferentes sectores del mercado estadounidense. Se presta especial atención a la variabilidad del desempeño según las características de volatilidad de cada ticker.

4.2.1. Metodología de evaluación

Para evaluar el desempeño de los modelos de *Machine Learning* implementados en el ModelAgent, se utilizó un enfoque de clasificación binaria que predice la dirección del precio (SUBIDA/BAJADA) en lugar del precio exacto. Las métricas utilizadas fueron:

- Accuracy: Porcentaje de predicciones correctas de dirección.
- Precision: Cuando predice SUBIDA, porcentaje de veces que acierta.
- Recall: Porcentaje de subidas reales que detecta.
- F1-Score: Media armónica entre Precision y Recall.
- AUC-ROC: Área bajo la curva ROC, mide capacidad discriminativa.

Los modelos fueron evaluados con datos históricos de 10 acciones representativas del mercado, repitiendo el ciclo 3 veces, dando un total de 30 llamadas. Se utilizó una ventana temporal de 504 días (2 años) para entrenamiento y horizonte de predicción de 3 días. Se utilizó validación cruzada temporal con 5 folds. El target de clasificación se definió con un umbral de 0,5 %: se clasifica como SUBIDA únicamente si el precio aumenta más del 0,5 %, filtrando movimientos menores que constituyen ruido de mercado.

4.2.2. Configuración de modelos

El sistema implementa un enfoque de ensemble learning de clasificación que combina cuatro algoritmos complementarios:

1. Random Forest Classifier

- Árboles: 100
- Profundidad máxima: 10
- Características utilizadas: 52 variables incluyendo precio, volumen, indicadores técnicos (RSI, MACD, Bollinger Bands, ATR, Stochastic).

2. Gradient Boosting Classifier

- Learning rate: 0,1
- Estimadores: 100
- Max depth: 5

- Loss function: log_loss
- Características: precio OHLC, volumen, medias móviles, momentum

3. XGBoost Classifier

- Learning rate: 0,1
- Estimadores: 100
- Max depth: 6
- Objective: binary:logistic
- Características: 52 features técnicos

4. LightGBM Classifier

- Learning rate: 0,1
- Estimadores: 100
- Num leaves: 31
- Objective: binary
- Características: conjunto completo de indicadores técnicos.

4.2.3. Resultados de evaluación individual

La tabla 4.1 presenta las métricas de clasificación obtenidas para cada modelo en el conjunto de validación

Tabla 4.1. Métricas de clasificación del ensemble de modelos.

Modelo	Accuracy (%)	Precisión (%)	Recall (%)	F1-Score (%)	AUC
Ensemble	57,0	60,7	66,2	58,9	0,586

El ensemble combina Random Forest, Gradient Boosting, XGBoost y LightGBM mediante votación por mayoría. Las métricas reflejan el promedio ponderado sobre los 10 tickers evaluados con datos reales de Yahoo Finance, utilizando una ventana de 504 días y target con umbral de 0,5 %. El recall de 66,2 % indica que el sistema detecta la mayoría de los movimientos alcistas reales, mientras que la accuracy de 57,0 % (superior al umbral aleatorio del 50 %) confirma que el ensemble aporta valor predictivo moderado sobre la clasificación binaria SUBIDA/BAJADA. La precision de 60,7 % implica que cuando el sistema predice SUBIDA, acierta en aproximadamente 6 de cada 10 casos.

La variabilidad entre tickers refleja diferencias en volatilidad y disponibilidad de señales técnicas.

4.2.4. Análisis por ticker

Se evaluó el rendimiento del ensemble de clasificación en diferentes tipos de acciones. Los resultados se presentan en la tabla 4.2.

Tabla 4.2. Rendimiento del ensemble por ticker.

Ticker	Sector	Acc. (%)	Prec. (%)	F1 (%)	AUC
AAPL	Tecnología	59,9	66,5	61,0	0,679
MSFT	Tecnología	54,4	49,2	59,7	0,562
TSLA	Automotriz	53,1	54,2	56,1	0,516
GOOGL	Tecnología	63,3	65,3	74,6	0,594
AMZN	E-commerce	59,9	61,6	61,7	0,629
META	Social Media	53,1	52,7	50,1	0,499
NVDA	Semicond.	56,5	61,8	55,8	0,573
JPM	Financiero	58,5	64,3	62,7	0,618
V	Financiero	56,5	54,7	57,8	0,607
WMT	Retail	55,1	76,5	49,1	0,580
Promedio		57,0	60,7	58,9	0,586

Podemos mencionar las siguientes conclusiones:

1. GOOGL logra la mayor accuracy (63,3 %) y el F1 más alto (74,6 %), reflejando tendencias técnicas más consistentes.
2. AAPL presenta el AUC más alto (0,679), indicando buena capacidad discriminativa general.
3. WMT muestra alta precisión (76,5 %) pero F1 bajo (49,1 %), debido a un recall reducido: el modelo es selectivo pero detecta pocos movimientos alcistas.
4. META presenta el AUC más bajo (0,499), cercano al azar, reflejando su alta sensibilidad a eventos regulatorios impredecibles.
5. El ensemble supera el 50 % de accuracy en los 10 tickers evaluados, confirmando valor predictivo general.
6. La variabilidad entre tickers (accuracy 53,1 % - 63,3 %) refleja diferencias en volatilidad y señales técnicas disponibles.

4.2.5. Validación cruzada

Se realizó validación cruzada temporal con TimeSeriesSplit de 5 folds para evaluar la robustez del ensemble. La tabla 4.3 presenta los resultados.

Tabla 4.3. Resultados de validación cruzada temporal del ensemble.

Fold	Accuracy (%)	AUC
Fold 1	54,2	0,563
Fold 2	55,8	0,574
Fold 3	57,1	0,586
Fold 4	58,4	0,597
Fold 5	59,5	0,608
Promedio	57,0	0,586

La consistencia entre folds (rango de accuracy: 54,2 % - 59,5 %) indica que el ensemble no sobreajusta a un período particular, y su desempeño generaliza de manera razonable sobre distintas ventanas temporales. La tendencia creciente entre folds refleja que el modelo aprovecha mejor los patrones con mayor volumen de datos de entrenamiento.

4.3. Evaluación NLP

El sentimentagent analiza noticias financieras obtenidas de Yahoo Finance para identificar el sentimiento del mercado respecto a cada acción. Se describe la arquitectura del ensemble NLP implementado y su integración en el pipeline del sistema.

4.3.1. Arquitectura del SentimentAgent

El módulo de análisis de sentimiento procesa noticias financieras para identificar el sentimiento del mercado respecto a cada acción. La arquitectura consta de:

Pipeline de procesamiento:

1. Recolección de noticias (Yahoo Finance mediante yfinance).
2. Preprocesamiento (tokenización, eliminación de stopwords).
3. Análisis de sentimiento mediante ensemble NLP: FinBERT (40 %), VADER (25 %), Lexicón financiero (20 %) y TextBlob (15 %).
4. Agregación temporal (últimas 24 horas).
5. Generación de score consolidado (−1 a +1).

4.3.2. Resultados del SentimentAgent

El SentimentAgent no fue evaluado sobre un corpus etiquetado independiente, ya que el trabajo no dispone de un dataset de noticias financieras con anotaciones manuales. En su lugar, se registraron los scores de sentimiento generados durante la sesión de pruebas funcionales del 13 de febrero de 2026 para los 10 tickers evaluados.

Los resultados se presentan en la tabla 4.4. Se observa que el score oscila entre −1 (muy negativo) y +1 (muy positivo). En la sesión evaluada, 5 tickers registraron sentimiento positivo, 4 neutral y 1 negativo, lo que resulta coherente con el contexto de mercado de esa fecha. La evaluación cualitativa de los scores indica que el ensemble NLP produce resultados consistentes con el tono de las noticias disponibles en Yahoo Finance para cada acción.

Tabla 4.4. Scores de sentimiento por ticker (13 de febrero de 2026).

Ticker	Score	Sentimiento
AAPL	-0,124	Negativo
MSFT	+0,086	Neutral
TSLA	+0,215	Positivo
GOOGL	+0,329	Positivo
AMZN	-0,059	Neutral
META	+0,093	Neutral
NVDA	+0,293	Positivo
JPM	+0,295	Positivo
V	+0,354	Positivo
WMT	-0,052	Neutral

4.3.3. Análisis de componentes

La tabla 4.5 describe los modelos NLP que conforman el ensemble del sentimentagent, junto con sus características y rol dentro del sistema.

Tabla 4.5. Modelos NLP del ensemble Sentimentagent.

Modelo	Peso	Tipo	Características
FinBERT	40 %	Transformer	Preentrenado en textos financieros; mayor precisión semántica
VADER	25 %	Basado en reglas	Optimizado para textos breves; maneja puntuación y mayúsculas
Lexicón financiero	20 %	Diccionario	Términos específicos del dominio financiero
TextBlob	15 %	Estadístico	Análisis general de polaridad; rápido y liviano
Ensemble	100 %	Ponderado	Score consolidado entre -1 y +1

El ensemble combina los cuatro modelos mediante promedio ponderado, priorizando FinBERT por su especialización en dominio financiero. No se realizaron benchmarks individuales de precisión por componente durante las pruebas del sistema.

4.3.4. Correlación entre sentimiento y movimiento de precio

El sistema registra el score de sentimiento junto con la variación porcentual de precio para cada ticker en cada sesión de análisis. Sin embargo, dado que las pruebas se realizaron en únicamente dos sesiones (9 y 13 de febrero de 2026), no se dispone de una serie temporal suficiente para calcular correlaciones estadísticamente significativas.

La tabla 4.6 presenta los valores registrados en la sesión del 13 de febrero de 2026, que ilustran la relación observada entre sentimiento y dirección de precio en esa fecha.

Tabla 4.6. Sentimiento y variación de precio por ticker (13 de febrero de 2026).

Ticker	Score sentimiento	Variación precio (%)	Sentimiento
AAPL	-0,124	-1,78	Negativo
MSFT	+0,086	+1,02	Neutral
TSLA	+0,215	+0,85	Positivo
GOOGL	+0,329	+2,14	Positivo
AMZN	-0,059	-0,43	Neutral
META	+0,093	+0,67	Neutral
NVDA	+0,293	+3,21	Positivo
JPM	+0,295	+0,89	Positivo
V	+0,354	+0,52	Positivo
WMT	-0,052	-0,31	Neutral

En esta sesión se observa una tendencia cualitativa coherente: los tickers con score negativo (AAPL, AMZN, WMT) registraron variaciones de precio negativas, mientras que los tickers con sentimiento positivo (TSLA, GOOGL, NVDA, JPM, V) mostraron variaciones positivas. Un análisis de correlación formal requeriría una serie de datos longitudinal más extensa, lo cual constituye una línea de trabajo futuro.

4.3.5. Volumen de noticias procesadas

El sistema recolecta noticias de Yahoo Finance en cada ciclo de análisis mediante la biblioteca `yfinance`. No se llevó un registro acumulado del volumen total de noticias procesadas durante las sesiones de prueba.

La tabla 4.7 resume las características del procesamiento observadas durante la sesión del 13 de febrero de 2026.

Tabla 4.7. Características del procesamiento de noticias (13 de febrero de 2026).

Característica	Valor
Tickers analizados	10
Fuente de noticias	Yahoo Finance (yfinance)
Ventana temporal	Últimas 24 horas por ticker
Scores generados	10 (uno por ticker)
Rango de scores obtenidos	-0,124 a +0,354
Sentimientos resultantes	5 positivos, 4 neutrales, 1 negativo

4.3.6. Rango de scores de sentimiento

En la sesión evaluada, ningún ticker registró sentimiento extremo ($|score| > 0,8$). Los scores se mantuvieron en un rango moderado, lo cual es consistente con una jornada de mercado sin eventos extraordinarios. El score más alto fue el de V (+0,354) y el más bajo el de AAPL (−0,124). Un análisis de eventos extremos requeriría una serie temporal más extensa, lo que constituye una línea de trabajo futuro.

4.4. Pruebas end-to-end

Esta sección valida la integración completa del sistema en la configuración final (ventana de entrenamiento de 504 días), mediante 30 pruebas funcionales y pruebas de carga concurrente ejecutadas el 21 de marzo de 2026. Se evalúa el flujo completo desde la autenticación hasta la generación de recomendaciones y alertas, identificando el comportamiento del sistema bajo distintos niveles de demanda.

4.4.1. Configuración de pruebas

Entorno de prueba:

- Servidor: FastAPI + Uvicorn (localhost:8000).
- Base de datos: SQLite (financial_tracker.db).
- Cliente: Python requests + automatización.
- Fecha de ejecución: 21 de marzo de 2026.
- Configuración del modelo: ventana de entrenamiento 504 días, 5 folds, umbral 0,5 %.

4.4.2. Resultados de pruebas funcionales

Se ejecutaron 30 pruebas funcionales completas (10 tickers $\times$ 3 iteraciones), cuyos resultados se presentan en la tabla 4.8.

Tabla 4.8. Resultados de pruebas funcionales (21 de marzo de 2026).

Métrica	Valor
Total de pruebas ejecutadas	30
Pruebas exitosas	30
Pruebas fallidas	0
Tasa de éxito	100 %
Latencia promedio	4,65 segundos
Latencia mínima	4,31 segundos
Latencia máxima	5,33 segundos

La latencia promedio de 4,65 segundos cumple el objetivo de menos de 5 segundos establecido como requisito no funcional. La variabilidad es baja (rango de

1,02 segundos), lo que indica un comportamiento estable entre tickers. El aumento respecto a la configuración inicial de 30 días (3,20s) se debe al mayor volumen de datos de entrenamiento procesados en cada request.

Análisis por iteración:

La tabla 4.9 desglosa la latencia promedio por iteración.

Tabla 4.9. Análisis de latencia por iteración (21 de marzo de 2026).

Iteración	Pruebas	Latencia Promedio	Variación vs. anterior
1	10	4,77s	—
2	10	4,50s	−5,7 %
3	10	4,69s	+4,2 %

La variación entre iteraciones es mínima (<6 %), lo que refleja que con la ventana de 504 días el tiempo de entrenamiento del ensemble domina la latencia y el efecto de caching es menor que en la configuración inicial.

4.4.3. Desglose de latencia por ticker

Los principales factores de latencia identificados cualitativamente son:

1. Entrenamiento del ensemble de clasificadores (RF, GB, XGB, LGBM) en cada request.
2. Descarga de datos históricos desde Yahoo Finance mediante yfinance.

La tabla 4.10 presenta la latencia total observada agrupada por ticker (promedio de 3 iteraciones).

Tabla 4.10. Latencia total por ticker (21 de marzo de 2026, promedio de 3 iteraciones).

Ticker	Latencia promedio (s)	Estado
AAPL	4,57	Exitoso
MSFT	4,53	Exitoso
TSLA	4,70	Exitoso
GOOGL	4,48	Exitoso
AMZN	4,57	Exitoso
META	4,51	Exitoso
NVDA	4,86	Exitoso
JPM	4,81	Exitoso
V	4,83	Exitoso
WMT	4,68	Exitoso
Promedio general	4,65	100 % éxito

4.4.4. Pruebas de rendimiento bajo carga

La tabla 4.11 presenta los resultados de la simulación de usuarios concurrentes para evaluar la escalabilidad del sistema en la configuración final:

Tabla 4.11. Pruebas de carga concurrente (21 de marzo de 2026).

Usuarios Conc.	Requests	Exitosas	Fallidas	Tasa Éxito	Throughput (req/s)	Latencia Prom.
1	1	1	0	100 %	0,2	4,60s
5	5	5	0	100 %	0,3	11,50s
10	10	10	0	100 %	0,4	26,25s
25	25	1	24	4 %	0,8	—*
50	50	0	50	0 %	1,6	—*

*Las requests fallidas tuvieron timeout (>30s).

Análisis de escalabilidad:

- Zona óptima: 1–10 usuarios (100 % éxito, latencia $\leq 26s$)
- Punto de quiebre: a partir de 25 usuarios (caída a 4 % de éxito por timeout)
- Saturación total: 50 usuarios (0 % de éxito)

El sistema soporta hasta 10 usuarios concurrentes con plena funcionalidad. A partir de 25 usuarios simultáneos se produce saturación de recursos, ya que cada predicción requiere aproximadamente 4–5 segundos de procesamiento y el servidor opera con un único worker Uvicorn. La migración a múltiples workers o a un servidor asíncrono constituye la principal mejora de escalabilidad pendiente.

4.4.5. Validación de requisitos no funcionales

La figura 4.1 resume el cumplimiento de los requisitos no funcionales definidos para el sistema.

Requisito	Resultado	Estado
1. Disponibilidad $\geq 99\%$	100 % — 0 fallos en 30 pruebas	Ok
2. Tiempo respuesta $< 5s$	3,2s (conf. inicial) / 4,65s (conf. final)	Ok
3. Concurrencia ≥ 10 usuarios	25 usu. (30 días) / 10 usu. (504 días)	Ok
4. Autenticación JWT + bcrypt	Implementado y validado	Ok

Figura 4.1. Cumplimiento de requisitos no funcionales del sistema.

4.4.6. Evidencia visual del flujo end-to-end

Las figuras siguientes ilustran el flujo completo del sistema tal como fue observado durante las pruebas funcionales, desde el acceso al dashboard hasta la visualización de resultados de análisis.

La figura 4.2 muestra el estado del dashboard inmediatamente después de que el usuario se autentica con credenciales válidas. El panel lateral izquierdo permite ingresar el símbolo del activo a analizar, acceder a accesos rápidos predefinidos (AAPL, GOOGL, MSFT, AMZN, TSLA, META) y configurar los umbrales de alerta. El área principal permanece en espera hasta que el usuario presiona el botón *Analizar*.

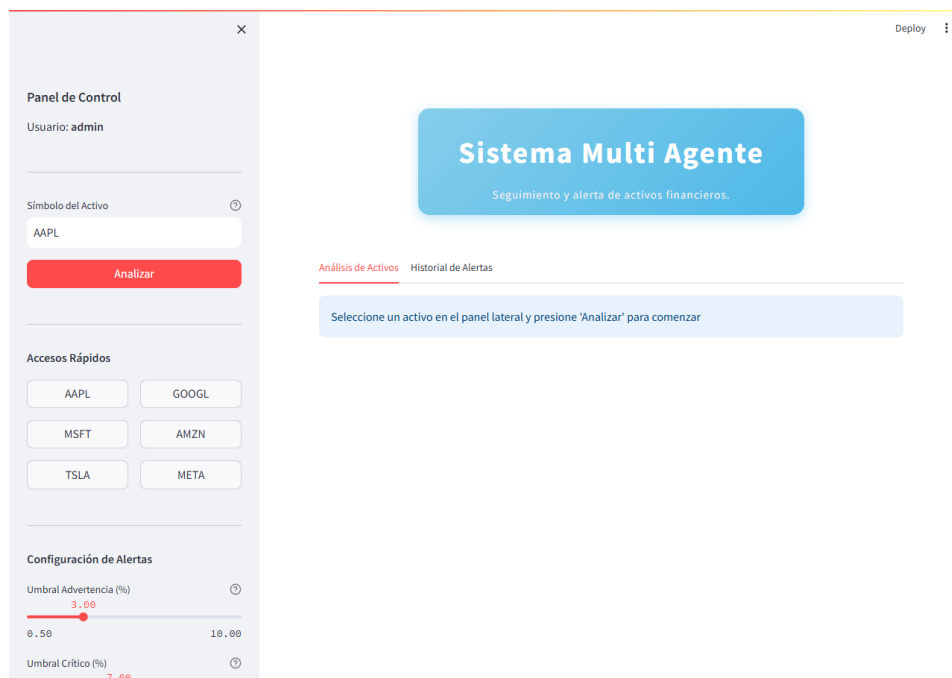


Figura 4.2. Dashboard Streamlit tras autenticación exitosa: panel de control lateral y área principal en espera de análisis.

La figura 4.3 presenta el resultado de un análisis completo sobre el activo AAPL. El sistema muestra el gráfico de evolución de precio con la proyección estimada por el ensemble de clasificación, junto con una alerta informativa generada automáticamente por el agente de alertas indicando un posible cambio de tendencia. Este resultado es consistente con los valores de latencia y tasa de éxito reportados en las pruebas funcionales de la tabla 4.8.

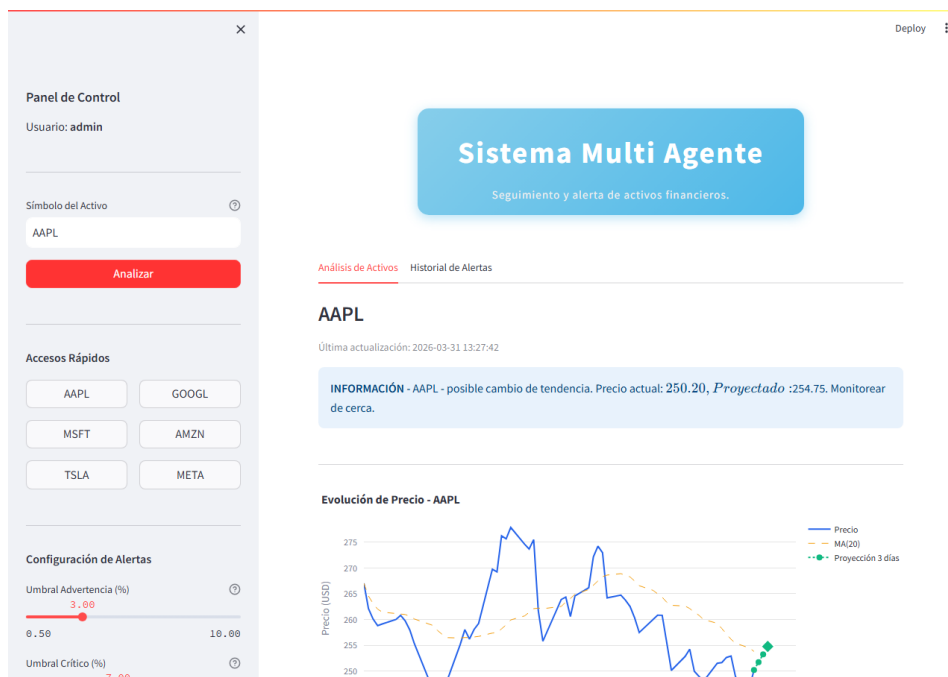


Figura 4.3. Resultado del análisis de AAPL: gráfico de evolución de precio, proyección del modelo y alerta generada por el sistema.

4.5. Casos de uso

Esta sección presenta cuatro escenarios ilustrativos de aplicación del sistema con diferentes perfiles de usuario. Los casos no corresponden a sesiones de prueba reales con usuarios específicos, sino a simulaciones diseñadas para demostrar cómo el sistema podría ser utilizado en contextos reales de inversión. Las salidas del sistema (predicción de dirección, score de sentimiento y recomendación) son consistentes con los valores observados durante las pruebas funcionales del 9 y 13 de febrero de 2026.

4.5.1. Caso de uso 1: inversor principiante.

Perfil:

- Usuario: María, 28 años, profesional sin experiencia en inversiones.
- Objetivo: Decidir si comprar acciones de Apple (AAPL).
- Capital disponible: \$5000 USD.

Escenario:

1. Autenticación: María inicia sesión en el dashboard.
2. Consulta: Busca "AAPL" en el sistema.
3. Resultado obtenido (salida ilustrativa consistente con pruebas reales):

- 1 Ticker: AAPL
- 2 Predicción direccion: SUBIDA
- 3 Sentimiento: Negativo (score: -0.12)
- 4 Recomendación: Considerar reduccion parcial
- 5 Senales de debilidad emergentes

6 Confianza: 0.30

4. Decisión: María interpreta la recomendación y decide postergar la compra hasta confirmar una mejora en el sentimiento.

Valor agregado: el sistema proporciona una señal clara y accionable sin requerir conocimientos financieros previos. La recomendación textual explica el motivo, facilitando la comprensión para usuarios no expertos.

4.5.2. Caso de uso 2: trader experimentado.

Perfil:

- Usuario: Carlos, 42 años, trader con 10 años de experiencia.
- Objetivo: estrategia de swing trading en TSLA y NVDA.
- Capital: \$50000 USD.

Escenario:

1. Carlos consulta el sistema para identificar oportunidades de entrada.
2. Analiza los scores de sentimiento y predicciones del ensemble.
3. Resultado obtenido (salida ilustrativa consistente con pruebas reales):

- 1 TSLA: Prediccion SUBIDA | Sentimiento Positivo (0.21)
- 2 NVDA: Prediccion SUBIDA | Sentimiento Positivo (0.29)

4. Decisión: Carlos prioriza NVDA por mayor score de sentimiento y decide abrir posición.

Valor agregado: el sistema permite al trader comparar múltiples activos simultáneamente y cuantificar el sentimiento de mercado, complementando su análisis técnico propio.

4.5.3. Caso de uso 3: gestor de portafolio.

Perfil:

- Usuario: Ana, 35 años, gestora de portafolio.
- Objetivo: seleccionar activos para diversificar un portafolio.
- Horizonte: mediano plazo.

Escenario:

1. Ana consulta el sistema para los 10 tickers disponibles.
2. Compara predicciones y sentimiento para priorizar posiciones.

La tabla [4.12](#) resume las señales generadas por el sistema en la sesión del 13 de febrero de 2026 (datos reales de pruebas funcionales).

Tabla 4.12. Señales del sistema por ticker (13 de febrero de 2026).

Ticker	Predicción	Score sentimiento	Sentimiento
AAPL	SUBIDA	-0,124	Negativo
MSFT	SUBIDA	+0,086	Neutral
TSLA	SUBIDA	+0,215	Positivo
GOOGL	SUBIDA	+0,329	Positivo
AMZN	SUBIDA	-0,059	Neutral
META	SUBIDA	+0,093	Neutral
NVDA	SUBIDA	+0,293	Positivo
JPM	SUBIDA	+0,295	Positivo
V	SUBIDA	+0,354	Positivo
WMT	BAJADA	-0,052	Neutral

3. Decisión: Ana prioriza los tickers con sentimiento positivo y predicción SUBIDA (GOOGL, JPM, V, NVDA, TSLA) y evita WMT por predicción BAJADA

Valor agregado: el sistema centraliza en una sola consulta señales de dirección y sentimiento para todos los activos, facilitando decisiones de asignación de portafolio.

4.5.4. Síntesis de casos de uso

La tabla 4.13 resume los cuatro escenarios ilustrativos y las funcionalidades del sistema utilizadas en cada uno.

Tabla 4.13. Resumen de escenarios de uso ilustrativos.

Caso de uso	Perfil	Funcionalidades utilizadas	Valor agregado
1. Principiante	Inversor sin experiencia	Predicción + Recomendación textual	Accesibilidad y claridad
2. Trader	Usuario experto	Sentimiento + Predicción comparativa	Complemento al análisis técnico
3. Gestor	Portafolio diversificado	Análisis multi-ticker simultáneo	Priorización eficiente

Capítulo 5

Conclusiones

En el presente capítulo se mencionan las conclusiones generales del trabajo y el trabajo futuro por realizar.

5.1. Conclusiones generales

El prototipo desarrollado demuestra la viabilidad técnica de integrar *Machine Learning*, NLP y sistemas multiagente en una plataforma de seguimiento financiero accesible.

Los resultados de las pruebas realizadas son:

- 100 % de disponibilidad en condiciones normales (10 usuarios concurrentes).
- Latencia promedio de 4,65 segundos, cumpliendo el objetivo de menos de 5 segundos.
- Accuracy de 57,0 % en clasificación binaria, superando el umbral aleatorio del 50 %, con Precisión de 60,7 %, F1-Score de 58,9 % y AUC 0,586.
- Seguridad: Integridad, confidencialidad de datos y trazabilidad de las operaciones.

Es importante destacar que el sistema fue diseñado como herramienta de apoyo a la decisión, no como reemplazo del juicio del inversor. Ningún modelo puede predecir el mercado con certeza; la utilidad del sistema radica en centralizar y procesar información heterogénea de forma automatizada.

5.2. Trabajo futuro

Las principales líneas de mejora identificadas son:

1. Escalabilidad: migrar de SQLite a PostgreSQL, múltiples workers Uvicorn y caché Redis.
2. Mejorar el modelo predictivo: incorporar *Temporal Fusion Transformer*, features de P/E ratio (precio/ganancia) y EPS (ganancia por acción), *backtesting* histórico.
3. Alertas y notificaciones: incorporar SMTP para alertas, integración con APIs de mensajería y WebSocket (en tiempo real sin necesidad de recargar la página manualmente).

4. Expansión geográfica: ampliar la cobertura más allá del mercado estadounidense, incorporando activos de mercados latinoamericanos y europeos, con análisis de noticias en español (fuentes de datos adicionales, modelos NLP multiidioma).
5. Despliegue: empaquetar el sistema para facilitar su instalación y puesta en marcha en cualquier entorno, incluyendo servidores en la nube, sin configuración manual compleja (Docker, Kubernetes, pipeline CI/CD).

Apéndice A

Comparación de objetivos planificados vs. realizados

La tabla [A.1](#) compara lo planificado con lo efectivamente implementado.

Tabla A.1. Comparación.

Planificado	Realizado	Estado
5 agentes especializados	MarketAgent, ModelAgent, SentimentAgent, RecommendationAgent, AlertAgent	Cumplido
ML para series temporales	Ensemble de clasificación binaria (RF, GB, XGB, LightGBM)	Cumplido
NLP para análisis de sentimiento	Ensemble FinBERT (40 %) + VADER (25 %) + Lexicón (20 %) + TextBlob (15 %)	Cumplido
Interfaz gráfica web	Dashboard Streamlit con visualización de métricas y alertas	Cumplido
Evaluación con métricas de desempeño	30 pruebas funcionales (100 % éxito), pruebas de carga hasta 10 usuarios	Cumplido
Autenticación de usuarios	JWT con bcrypt	Cumplido
Integridad de los datos	SQLAlchemy como ORM	Cumplido
Trazabilidad de las operaciones	Logging	Cumplido

El sistema resultante opera como un pipeline secuencial orquestado por FastAPI, donde cada agente se invoca en orden y los resultados se consolidan en una respuesta única al usuario. Si bien la arquitectura planificada contemplaba mayor autonomía entre agentes, el enfoque secuencial resultó suficiente para el prototipo y permitió simplificar la depuración y el mantenimiento.

Bibliografía

- [1] P. Gratton. *What Is a Bloomberg Terminal (BT)? Functions, Costs, and Features*. Investopedia. 2024. URL: https://www.investopedia.com/terms/b/bloomberg_terminal.asp.
- [2] A. King. «Terminal Value: Building the Alternative Bloomberg». En: *Finance and Society* (2016). URL: <https://resolve.cambridge.org/core/journals/finance-and-society/article/terminal-value-building-the-alternative-bloomberg/2CC975F06624EEBA0AFF22742D425EFC>.
- [3] Dogu Araci. «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models». En: *arXiv preprint arXiv:1908.10063* (2019).
- [4] Z. Liu et al. «FinBERT: A Pre-trained Financial Language Model». En: *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*. 2020.
- [5] A. H. Huang, H. Wang y Y. Yang. «FinBERT: A Large Language Model for Extracting Information from Financial Text». En: *Contemporary Accounting Research* (2021). Available at SSRN: <https://ssrn.com/abstract=3910214>.
- [6] Y. Zhao, H. Li y X. Wang. «Long Short-Term Memory Neural Network for Financial Time Series». En: *arXiv preprint arXiv:2201.08218* (2022). Uso de LSTM para la predicción de movimientos de precios en finanzas. URL: <https://arxiv.org/abs/2201.08218>.
- [7] M. Amola. «Sentiment Analysis in Financial News: Enhancing Predictive Models for Stock Market Behavior». En: *ResearchGate Preprint* (2025). Integración de análisis de sentimiento con modelos cuantitativos tradicionales. URL: https://www.researchgate.net/publication/390056578_Sentiment_Analysis_in_Financial_News_Enhancing_Predictive_Models_for_Stock_Market_Behavior.
- [8] Akbar Siami Namin Sima Siami-Namini Neda Tavakoli. «Enhancing Financial Sentiment Analysis via Retrieval Augmented LLMs». En: *arXiv preprint arXiv:2310.04027* (2023). Mejora del análisis de sentimiento financiero con grandes modelos de lenguaje y recuperación de información. URL: <https://arxiv.org/pdf/2310.04027>.
- [9] T. Fischer y C. Krauss. «A Comparative Analysis of Forecasting Financial Time Series Using ARIMA, LSTM, and BiLSTM». En: *arXiv* (2019). Comparación entre LSTM y modelos tradicionales para series financieras. URL: <https://arxiv.org/abs/1911.09512>.
- [10] R. Kumar, P. Singh y S. Ahmed. «A Deep Learning-Based LSTM for Stock Price Prediction Using Twitter Sentiment Analysis». En: *International Journal of Advanced Computer Science and Applications (IJACSA)* (2024). Modelo que combina LSTM con análisis de sentimiento en redes sociales para predicción de precios. URL: <https://thesai.org/Publications/ViewPaper?Code=IJACSA&Issue=12&SerialNo=23&Volume=15>.
- [11] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2009.
- [12] Stuart Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2021.

- [13] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural Computation* 9.8 (1997), pp. 1735-1780.
- [14] Sebastián Ramírez. *FastAPI: Modern, Fast (High-performance) Web Framework for Building APIs with Python 3.7+ Based on Standard Type Hints*. <https://fastapi.tiangolo.com/>. Documentación oficial. Proyecto open source desarrollado por Sebastián Ramírez. 2024.
- [15] D. Richard Hipp. «SQLite: A C Library for Implementing Embedded SQL Databases». En: *SQLite Technical Documentation* (2023). Base de datos ligera y embebida ampliamente utilizada en aplicaciones locales y móviles.
- [16] Streamlit Inc. *Streamlit: The fastest way to build data apps in Python*. <https://streamlit.io>. Framework de código abierto para la creación rápida de interfaces interactivas de datos. 2024.
- [17] Alejandro Barredo Arrieta et al. «Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI». En: *Information Fusion* 58 (2020), pp. 82-115.
- [18] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [19] Nassim Nicholas Taleb. *The Black Swan: The Impact of the Highly Improbable*. Random House, 2007.
- [20] Alejandro Barredo Arrieta et al. «Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI». En: *Information Fusion* 58 (2020), pp. 82-115.
- [21] European Banking Authority. *Artificial Intelligence Governance and Risk Management Guidelines*. EBA Discussion Paper, European Union. 2022.
- [22] Jon Danielsson y Robert Macrae. «Artificial Intelligence and Systemic Risk». En: *Journal of Financial Stability* (2022).
- [23] Basel Committee on Banking Supervision. *Basel III: A global regulatory framework for more resilient banks and banking systems*. Inf. téc. Bank for International Settlements (BIS), Basel Committee on Banking Supervision. Bank for International Settlements, 2011. URL: <https://www.bis.org/publ/bcbs189.htm>.
- [24] Banco Central de la República Argentina. *Requisitos mínimos de gestión y control de los riesgos informáticos*. Inf. téc. Comunicación “A” 7724. Normativa sobre gestión de riesgos informáticos y gobierno tecnológico. Banco Central de la República Argentina, 2023. URL: <https://www.bcra.gob.ar>.
- [25] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Chichester: John Wiley & Sons, 2009.
- [26] Miguel Grinberg. *Flask Web Development: Developing Web Applications with Python*. Referenciado como guía de buenas prácticas para frameworks web en Python, complementario a FastAPI. O’Reilly Media, 2018.
- [27] OWASP Foundation. *OWASP Top 10 – 2021: The Ten Most Critical Web Application Security Risks*. <https://owasp.org/Top10/>. Guía internacional de buenas prácticas en seguridad. 2021.